



Πανεπιστήμιο Μακεδονίας
Σχολή Επιστημών Πληροφορίας
Τμήμα Εφαρμοσμένης Πληροφορικής

TECHNICAL DEBT TRACEABILITY MAP

Πτυχιακή Εργασία
του
Κωνσταντίνου Θωμασιάδη

Θεσσαλονίκη, Ιούλιος 2024

TECHNICAL DEBT TRACEABILITY MAP

Κωνσταντίνος Θωμασιάδης

Προπτυχιακός φοιτητής στην Εφαρμοσμένη Πληροφορική του Πανεπιστημίου
Μακεδονίας

Πτυχιακή Εργασία

υποβάλλεται για τη μερική εκπλήρωση των απαιτήσεων του

ΠΤΥΧΙΟΥ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

Υπεύθυνος καθηγητής

Απόστολος Αμπατζόγλου

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή στις 09/07/2024

Αλέξανδρος Χατζηγεωργίου Απόστολος Αμπατζόγλου Στυλιανός Ξυνόγαλος

.....

.....

.....

Κωνσταντίνος Θωμασιάδης

.....

Περίληψη

Το τεχνικό χρέος είναι ένα από τα πιο κρίσιμα ζητήματα όταν μιλάμε για την ανάπτυξη ενός λογισμικού. Αντιπροσωπεύει τις μακροπρόθεσμες συνέπειες των μη βέλτιστων αποφάσεων που λαμβάνονται για βραχυπρόθεσμα οφέλη. Η παρούσα διατριβή παρουσιάζει την ανάπτυξη ενός εργαλείου λογισμικού που έχει σχεδιαστεί για τη διαχείριση και την οπτικοποίηση του τεχνικού χρέους με τη χρήση χαρτών ιχνηλασιμότητας. Κατασκευασμένο ως μια εφαρμογή πλήρους στοίβας με την χρήση της React και της Spring Boot, το εργαλείο ενσωματώνει προηγμένες τεχνικές οπτικοποίησης και φιλικές προς το χρήστη διεπαφές για να βοηθήσει τις ομάδες λογισμικού στη διατήρηση βιώσιμων βάσεων κώδικα. Η έρευνα περιλαμβάνει λεπτομερή επισκόπηση της υπάρχουσας βιβλιογραφίας, την εφαρμογή διαφόρων σύγχρονων τεχνολογιών και την υλοποίηση μιας ολοκληρωμένης μελέτης περίπτωσης. Τα ευρήματα καταδεικνύουν ότι το εργαλείο βελτιώνει σημαντικά τη διαχείριση και την οπτικοποίηση του τεχνικού χρέους, προσφέροντας πολύτιμες γνώσεις τόσο για ακαδημαϊκές όσο και για πρακτικές εφαρμογές.

***Λέξεις κλειδιά:* Technical Debt, Code Quality, Software Traceability, Traceability Maps, Software Maintenance, Software Development, Software Engineering, Java, Spring Boot, React, Neo4j, SonarQube, Visualization, Software Architecture, Full-Stack Development, Agile Methodologies, Technical Debt Management, Software Documentation, Code Analysis, Development Tools**

Περιεχόμενα

1. Εισαγωγή	14
1.1. Το πρόβλημα	14
1.2. Σημαντικότητα του προβλήματος	15
1.3. Σκοπός εργασίας.....	15
1.4. Συνεισφορά	15
2. Βιβλιογραφική επισκόπηση	17
2.1. Πληροφορίες υποβάθρου	17
2.1.1. Τεχνικό Χρέος.....	17
2.1.2. Software Traceability	22
2.1.3. Software Traceability Maps.....	24
2.2. Σχετική έρευνα	26
3. Μεθοδολογίες και Εργαλεία.....	27
3.1. Java.....	27
3.2. Spring Boot	30
3.3. Neo4j	30
3.4. PostgreSQL	32
3.5. JavaScript	33
3.6. TypeScript	35
3.7. React.....	36
3.8. Node.js.....	37
3.9. SonarQube.....	38
3.10. Docker	39
3.11. Git	40
4. Ανάπτυξη του εργαλείου	41

4.1.	Αρχιτεκτονική σχεδίαση	41
4.2.	Ανάπτυξη Backend.....	43
4.2.1.	Server Layer.....	43
4.2.2.	Data Layer.....	53
4.3.	Ανάπτυξη Frontend	55
5.	Επίλογος	65
6.	Αναφορές	66

Πίνακας εικόνων

Figure 1: Java programming language logo.....	27
Figure 2: Structure of Object-Oriented Programming.....	28
Figure 3: Java Virtual Machine architecture.....	29
Figure 4: Java programming language popularity chart.....	29
Figure 5: Spring Boot logo.....	30
Figure 6: Neo4j logo.....	30
Figure 7: DBMS popularity chart.....	31
Figure 8: PostgreSQL logo.....	32
Figure 9: PostgreSQL ranking chart.....	33
Figure 10: JavaScript logo.....	33
Figure 11: TypeScript logo.....	35
Figure 12: StackOverflow programming language ranking.....	36
Figure 13: React JS logo.....	36
Figure 14: Node.js logo.....	37
Figure 15: SonarQube logo.....	38
Figure 16: Docker logo.....	39
Figure 17: Git logo.....	40
Figure 18: Tool architecture.....	42
Figure 19: Project initialization Sequence Diagram.....	43
Figure 20: Spring Boot application's directory structure.....	45
Figure 21: Spring Boot application's design pattern.....	46
Figure 22: Spring Boot application's Models directory.....	47
Figure 23: Spring Boot application's Services directory.....	50
Figure 24: Spring Boot application's web endpoints.....	51
Figure 25: Neo4j database schema.....	54
Figure 26: React JS application's Home page 1.....	56
Figure 27: React JS application's Home page 2.....	56
Figure 28: React JS application's Analysis page 1.....	58
Figure 29: React JS application's Analysis page 2.....	59

Figure 30: React JS application's source directory structure	60
Figure 31: React JS index.tsx file	61
Figure 32: React JS App.tsx file	62
Figure 33: React JS virtual DOM	64

1. Εισαγωγή

Στον ταχέως εξελισσόμενο κόσμο της ανάπτυξης λογισμικού, η διαχείριση του τεχνικού χρέους αποτελεί μια κρίσιμη πρόκληση που αντιμετωπίζουν οι ομάδες ανάπτυξης. Το τεχνικό χρέος, που προκύπτει από την υιοθέτηση βραχυπρόθεσμων λύσεων εις βάρος της μακροπρόθεσμης ποιότητας του κώδικα, μπορεί να έχει σημαντικές επιπτώσεις στην αποδοτικότητα, το κόστος και τη βιωσιμότητα των έργων λογισμικού. Η παρούσα εργασία εστιάζει στην ανάπτυξη ενός καινοτόμου εργαλείου για τη διαχείριση του τεχνικού χρέους, αξιοποιώντας σύγχρονες τεχνολογίες και μεθοδολογίες. Το εργαλείο αυτό, που βασίζεται σε React και Spring Boot, προσφέρει μια ολοκληρωμένη λύση για την παρακολούθηση, την οπτικοποίηση και τη διαχείριση του τεχνικού χρέους μέσω χαρτών ιχνηλασιμότητας. Στις επόμενες ενότητες, θα αναλύσουμε λεπτομερώς το πρόβλημα του τεχνικού χρέους, τη σημασία του για τις επιχειρήσεις, τους στόχους του εργαλείου που αναπτύχθηκε, καθώς και τη συμβολή της έρευνας αυτής στον τομέα της ανάπτυξης λογισμικού. Μέσα από αυτή την ανάλυση, θα αναδείξουμε πώς το προτεινόμενο εργαλείο μπορεί να βοηθήσει τις ομάδες ανάπτυξης να διατηρούν υψηλής ποιότητας, βιώσιμες βάσεις κώδικα, βελτιώνοντας παράλληλα την παραγωγικότητα και μειώνοντας το κόστος συντήρησης.

1.1. Το πρόβλημα

Κάθε έργο λογισμικού καλείται να αντιμετωπίσει από την πρώτη στιγμή το πρόβλημα του τεχνικού χρέους. Το τεχνικό χρέος είναι μια έννοια που χρησιμοποιείται στην ανάπτυξη λογισμικού για να περιγράψει τις συνέπειες που προκύπτουν από την επιλογή βραχυπρόθεσμων λύσεων που διευκολύνουν την άμεση ανάπτυξη του λογισμικού, αλλά δημιουργούν πρόσθετα προβλήματα ή απαιτήσεις για την επίλυση μελλοντικών τεχνικών ζητημάτων. Κατά την διάρκεια της ζωής ενός έργου λογισμικού το τεχνικό χρέος έχει την τάση να αυξάνεται, ειδικότερα στα πρώτα στάδια της ανάπτυξής του όπου απαιτείται ταχύτερη παραγωγή. Αυτό έχει ως αποτέλεσμα να δυσχεραίνει την μετέπειτα ανάπτυξή του καθώς και την συντήρησή του. Το τεχνικό χρέος δημιουργείται από διάφορους παράγοντες οι οποίοι έχουν ένα κοινό, την μη χρήση των βέλτιστων πρακτικών ανάπτυξης. Συνεπώς, το τεχνικό χρέος μπορεί να δημιουργηθεί σε οποιοδήποτε στάδιο της ανάπτυξης με την κακή αρχιτεκτονική σχεδίαση ή την ελλιπή τεκμηρίωση ή και την έλλειψη ελέγχων και δοκιμών.

1.2. Σημαντικότητα του προβλήματος

Οι επιπτώσεις των βιαστικών αυτών πρακτικών ερμηνεύονται σε δύο πολύ σημαντικές διαστάσεις για τις επιχειρήσεις, τον χρόνο και το χρήμα. Καθώς το λογισμικό γίνεται πιο περίπλοκο και οι ανάγκες τις αγοράς περισσότερες, το τεχνικό χρέος δυσκολεύει ακόμα περισσότερο την ανάπτυξη και προσθήκη νέων λειτουργιών ή την συντήρηση των ήδη υπάρχοντων μειώνοντας την παραγωγικότητα και αυξάνοντας το κόστος συντήρησης και ανάπτυξης. Παρά τα σαφή οφέλη, πολλά υπάρχοντα εργαλεία ιχνηλασιμότητας δεν επαρκούν για την αποτελεσματική διαχείριση και οπτικοποίηση του τεχνικού χρέους. Αυτό το κενό αναδεικνύει την ανάγκη για εξειδικευμένα εργαλεία που παρέχουν ισχυρή υποστήριξη για τη διαχείριση του τεχνικού χρέους.

1.3. Σκοπός εργασίας

Το εργαλείο που αναπτύχθηκε στο πλαίσιο της παρούσας διατριβής στοχεύει στην παροχή εξειδικευμένων λειτουργιών για την διαχείριση του τεχνικού χρέους ενός έργου λογισμικού. Χτισμένο ως μια εφαρμογή πλήρους στοίβας που χρησιμοποιεί React για το frontend και SpringBoot για το backend, το εργαλείο αυτό προσφέρει ολοκληρωμένες δυνατότητες για τη δημιουργία, τη διαχείριση και την οπτικοποίηση χαρτών ιχνηλασιμότητας με ιδιαίτερη έμφαση στο τεχνικό χρέος. Ενσωματώνοντας προηγμένες τεχνικές οπτικοποίησης και φιλικές προς τον χρήστη διεπαφές, το εργαλείο ενισχύει την ικανότητα των ομάδων λογισμικού να διατηρούν υψηλής ποιότητας, βιώσιμες βάσεις κώδικα.

1.4. Συνεισφορά

Η έρευνα και το εργαλείο που αναπτύχθηκε στο πλαίσιο αυτής της πτυχιακής εργασίας προσφέρουν σημαντική συμβολή στη διαχείριση του τεχνικού χρέους στις διαδικασίες ανάπτυξης λογισμικού. Με τη χρήση των χαρτών ιχνηλασιμότητας (traceability maps), το εργαλείο επιτρέπει στις ομάδες ανάπτυξης να εντοπίζουν, να οπτικοποιούν και να διαχειρίζονται καλύτερα το τεχνικό χρέος, βελτιώνοντας τη συνολική ποιότητα και βιωσιμότητα του κώδικα. Η έρευνα προσθέτει στη βιβλιογραφία νέες μεθόδους για τη διαχείριση του τεχνικού χρέους μέσω σύγχρονων εργαλείων

και τεχνολογιών. Το εργαλείο ενσωματώνει προηγμένες τεχνολογίες όπως React για το frontend και Spring Boot για το backend, καθιστώντας το σύγχρονο και ευέλικτο στη χρήση. Επιπλέον, η ενσωμάτωση της βάσης δεδομένων Neo4j και του εργαλείου ανάλυσης κώδικα SonarQube προσφέρει μια ολοκληρωμένη λύση για την ανάλυση και τη διαχείριση του τεχνικού χρέους. Αυτή η καινοτομία επιτρέπει στους προγραμματιστές να παρακολουθούν τις επιπτώσεις των αποφάσεων ανάπτυξης σε πραγματικό χρόνο και να λαμβάνουν μέτρα για τη μείωση του τεχνικού χρέους. Τέλος, η εφαρμογή ενός συγκεκριμένου case study στην εργασία επιβεβαιώνει την πρακτική αξία του εργαλείου, επιδεικνύοντας τη λειτουργικότητά του σε πραγματικές συνθήκες. Η έρευνα παρέχει ένα χρήσιμο πλαίσιο για την ανάπτυξη παρόμοιων εργαλείων στο μέλλον και ανοίγει το δρόμο για περαιτέρω έρευνα και βελτίωση των τεχνικών διαχείρισης τεχνικού χρέους.

2. Βιβλιογραφική επισκόπηση

Στην ενότητα αυτή, θα παρουσιαστούν αναλυτικά οι έννοιες και το γνωστικό υπόβαθρο που χρησιμοποιούνται στην παρούσα εργασία. Θα γίνει αναλυτική περιγραφή των όρων «τεχνικό χρέος» και «χάρτης ιχνηλασιμότητας» καθώς και της επιρροής αυτών σε διάφορες πτυχές της ανάπτυξης και της συντήρησης ενός έργου λογισμικού. Τέλος, γίνεται αναφορά βιβλιογραφικών ερευνών σχετικά με το ερευνητικό ενδιαφέρον αυτής της εργασίας.

2.1. Πληροφορίες υποβάθρου

2.1.1. Τεχνικό Χρέος

Ο όρος «τεχνικό χρέος» αναφέρεται στις συνέπειες των αποφάσεων που λαμβάνονται κατά την ανάπτυξη λογισμικού, οι οποίες επιλέγουν την πιο γρήγορη ή εύκολη λύση σε βάρος της περισσότερο βέλτιστης ή μακροπρόθεσμα αποδοτικής λύσης. Η έννοια αυτή εισήχθη από τον Ward Cunningham το 1992 [9], με σκοπό την ερμηνεία και την κατανόηση των επιπτώσεων που προκύπτουν από ένα κακώς ανεπτυγμένο έργο λογισμικού από τους υπόλοιπους μη τεχνικούς ανθρώπους. Ο Cunningham παρομοίασε την επιλογή μιας πρόχειρης λύσης στον κώδικα με τη λήψη ενός δανείου: «μπορεί να προσφέρει άμεσα οφέλη, αλλά δημιουργεί υποχρεώσεις που πρέπει να εξοφληθούν αργότερα με τη μορφή πρόσθετης εργασίας για τη διόρθωση του κώδικα». Το τεχνικό χρέος, όπως το οικονομικό χρέος, μπορεί να συσσωρεύσει "τόκους" με τη μορφή αυξανόμενης δυσκολίας στην αλλαγή και συντήρηση του λογισμικού στο μέλλον. Τα αίτια για την συσσώρευση ποικίλουν και χωρίζονται σε διάφορους διακριτές κατηγορίες. Αρχικά, οι κατηγορίες διαχωρισμού του τεχνικού χρέους βασίζονται στον τρόπο δημιουργίας του, συνειδητά ή ασυνειδητά. Το σκόπιμο χρέος προκύπτει από συνειδητές αποφάσεις που λαμβάνονται σε κάθε στάδιο της παραγωγής ενός έργου λογισμικού.

Η φάση ανάλυσης απαιτήσεων αποτελεί το εναρκτήριο και θεμελιώδες στάδιο στον κύκλο ζωής ανάπτυξης λογισμικού, κατά το οποίο προσδιορίζονται, καταγράφονται και τεκμηριώνονται οι λειτουργικές και μη λειτουργικές απαιτήσεις του συστήματος. Οι αναλυτές συστημάτων

συνεργάζονται στενά με τους ενδιαφερόμενους φορείς για να κατανοήσουν πλήρως τις ανάγκες και τις προσδοκίες τους, χρησιμοποιώντας τεχνικές όπως συνεντεύξεις, ομάδες εστίασης και ανάλυση υφιστάμενων συστημάτων. Το κύριο παραδοτέο αυτής της φάσης είναι ένα ολοκληρωμένο έγγραφο προδιαγραφών απαιτήσεων, το οποίο χρησιμεύει ως σημείο αναφοράς για όλες τις επόμενες φάσεις ανάπτυξης. Ωστόσο, η φάση ανάλυσης απαιτήσεων μπορεί να συμβάλει στη δημιουργία τεχνικού χρέους με διάφορους τρόπους. Ένας σημαντικός παράγοντας είναι η ανεπαρκής ή ασαφής καταγραφή απαιτήσεων. Η ελλιπής ή διφορούμενη διατύπωση των απαιτήσεων μπορεί να οδηγήσει σε παρερμηνείες και λανθασμένες υλοποιήσεις στα επόμενα στάδια, δημιουργώντας την ανάγκη για εκτεταμένες αναθεωρήσεις και επανασχεδιασμούς. Για παράδειγμα, μια ασαφής περιγραφή των απαιτήσεων ασφαλείας μπορεί να οδηγήσει σε ανεπαρκή υλοποίηση μηχανισμών προστασίας, καθιστώντας το σύστημα ευάλωτο σε κινδύνους. Επιπλέον, η παράλειψη σημαντικών απαιτήσεων αποτελεί άλλη μια πηγή τεχνικού χρέους σε αυτή τη φάση. Η μη συμπερίληψη κρίσιμων λειτουργικών ή μη λειτουργικών απαιτήσεων μπορεί να οδηγήσει σε σημαντικές αναθεωρήσεις και ανακατασκευές σε μεταγενέστερα στάδια. Για παράδειγμα, η παράλειψη απαιτήσεων σχετικών με την κλιμάκωση του συστήματος μπορεί να οδηγήσει σε σημαντικές αρχιτεκτονικές αλλαγές όταν το σύστημα αντιμετωπίσει αυξημένο φόρτο εργασίας στο μέλλον.

Η φάση σχεδίασης είναι ένα κρίσιμο στάδιο στην ανάπτυξη λογισμικού, όπου οι προγραμματιστές μετατρέπουν τις απαιτήσεις του έργου σε ένα λεπτομερές σχέδιο για την υλοποίηση του συστήματος. Κατά τη διάρκεια αυτής της φάσης, η ομάδα ανάπτυξης καθορίζει την αρχιτεκτονική του λογισμικού, σχεδιάζει τη δομή των δεδομένων, επιλέγει τις κατάλληλες τεχνολογίες και εργαλεία, και δημιουργεί πρωτότυπα για να επικυρώσει τις ιδέες τους. Ο στόχος είναι να δημιουργηθεί ένα ολοκληρωμένο σχέδιο που θα καθοδηγήσει την επόμενη φάση της ανάπτυξης, διασφαλίζοντας ότι το τελικό προϊόν θα ανταποκρίνεται στις ανάγκες των χρηστών και θα είναι αποτελεσματικό, κλιμακούμενο και εύκολο στη συντήρηση. Ωστόσο, η φάση σχεδίασης μπορεί επίσης να συμβάλει σημαντικά στη δημιουργία τεχνικού χρέους, εάν δεν εκτελεστεί σωστά. Οι αρχιτεκτονικές ελλείψεις, όπως η μη τήρηση των αρχών σχεδιασμού, μπορούν να οδηγήσουν σε πολύπλοκα, δύσκολα συντηρήσιμα συστήματα. Για παράδειγμα, η παραβίαση αρχών όπως ο διαχωρισμός των ανησυχιών (separation of concerns) ή η αρχή της μοναδικής ευθύνης (single

responsibility principle) μπορεί να δημιουργήσει συστήματα με υψηλή συνοχή και χαμηλή συνεκτικότητα, καθιστώντας δύσκολη τη μελλοντική επέκταση ή τροποποίηση. Επιπλέον, η κακή οργάνωση μονάδων αποτελεί άλλη μια πηγή τεχνικού χρέους που μπορεί να προκύψει κατά τη φάση σχεδίασης. Η ανεπαρκής οργάνωση των ενοτήτων ή των συστατικών στοιχείων μπορεί να καταστήσει το σύστημα δύσκολο να κατανοηθεί και να επεκταθεί. Αυτό μπορεί να οδηγήσει σε αυξημένο χρόνο ανάπτυξης για μελλοντικές λειτουργίες, δυσκολίες στην εντοπισμό και διόρθωση σφαλμάτων, και σε ένα σύστημα που είναι ευάλωτο σε ακούσιες παρενέργειες κατά την εισαγωγή αλλαγών.

Η φάση ανάπτυξης αποτελεί κρίσιμο στάδιο στον κύκλο ζωής ανάπτυξης λογισμικού, κατά το οποίο το προκαταρκτικό σχέδιο μεταφράζεται σε λειτουργικό κώδικα. Οι προγραμματιστές, εφαρμόζοντας τις προδιαγραφές που έχουν καθοριστεί στις προηγούμενες φάσεις, προβαίνουν στην υλοποίηση των απαιτούμενων λειτουργιών και χαρακτηριστικών. Η διαδικασία αυτή διέπεται από καθιερωμένες μεθοδολογίες κωδικοποίησης και υποστηρίζεται από επιλεγμένα εργαλεία και πλαίσια ανάπτυξης. Παράλληλα, εφαρμόζονται τεχνικές διασφάλισης ποιότητας, όπως ο έλεγχος μονάδων, με στόχο την παραγωγή ενός άρτιου και λειτουργικού προϊόντος που ανταποκρίνεται στις προκαθορισμένες προδιαγραφές. Ωστόσο, η φάση ανάπτυξης ενέχει τον κίνδυνο δημιουργίας τεχνικού χρέους, το οποίο μπορεί να προκύψει μέσω διαφόρων μηχανισμών. Ένας καίριος παράγοντας είναι η ποιότητα του παραγόμενου κώδικα. Η ανεπαρκής τεκμηρίωση, η μη συμμόρφωση με τα καθιερωμένα πρότυπα κωδικοποίησης, ή η δημιουργία πολύπλοκων και δυσνόητων δομών δύνανται να οδηγήσουν σε δυσχέρειες κατά τη μελλοντική συντήρηση και επέκταση του λογισμικού. Για παράδειγμα, η έλλειψη επαρκούς τεκμηρίωσης δύνανται να καταστήσει προβληματική την κατανόηση και τροποποίηση του κώδικα από μεταγενέστερους προγραμματιστές, αυξάνοντας κατ' επέκταση τον απαιτούμενο χρόνο και κόστος για μελλοντικές τροποποιήσεις. Επιπροσθέτως, οι χρονικοί περιορισμοί και η πίεση για ταχεία παράδοση δύνανται να οδηγήσουν σε συμβιβασμούς ως προς την ποιότητα του κώδικα. Οι προγραμματιστές ενδέχεται να καταφύγουν σε προσωρινές λύσεις ή "hacks" προκειμένου να ανταποκριθούν στις προθεσμίες, παραβλέποντας τις βέλτιστες πρακτικές και συσσωρεύοντας τεχνικό χρέος. Χαρακτηριστικά παραδείγματα αποτελούν η παράλειψη επαρκούς ελέγχου μονάδων ή η αγνόηση των αρχών

καθαρού κώδικα, τα οποία δύνανται να οδηγήσουν σε ένα προϊόν που, ενώ λειτουργεί βραχυπρόθεσμα, παρουσιάζει δυσκολίες στη μακροπρόθεσμη συντήρηση και επέκτασή του.

Η φάση των δοκιμών αποτελεί κρίσιμο στάδιο στον κύκλο ζωής ανάπτυξης λογισμικού, κατά το οποίο το παραχθέν λογισμικό υποβάλλεται σε ενδελεχή έλεγχο προς διασφάλιση της ποιότητας και της λειτουργικότητάς του. Κατά τη διάρκεια αυτής της φάσης, εφαρμόζεται ένα ευρύ φάσμα μεθοδολογιών ελέγχου, συμπεριλαμβανομένων των δοκιμών μονάδας, των δοκιμών ολοκλήρωσης και των δοκιμών από άκρο σε άκρο. Οι ειδικοί ελέγχου λογισμικού διεξάγουν συστηματικούς ελέγχους για τον εντοπισμό σφαλμάτων, ασυνεπειών και αποκλίσεων από τις προδιαγραφές. Ο πρωταρχικός στόχος είναι η παράδοση ενός αξιόπιστου, ασφαλούς και λειτουργικού προϊόντος που ανταποκρίνεται πλήρως στις απαιτήσεις των χρηστών και τις προδιαγραφές του έργου. Ωστόσο, η φάση των δοκιμών δύναται να συμβάλει στη συσσώρευση τεχνικού χρέους, ιδιαίτερα υπό τη μορφή του χρέους δοκιμών. Ένας σημαντικός παράγοντας που συντελεί στη δημιουργία αυτού του χρέους είναι η ανεπαρκής δοκιμή. Η ελλιπής εφαρμογή δοκιμών μονάδας, δοκιμών ολοκλήρωσης ή δοκιμών από άκρο σε άκρο ενδέχεται να οδηγήσει στην ανάπτυξη εύθραυστων και αναξιόπιστων συστημάτων. Για παράδειγμα, η ανεπαρκής κάλυψη των δοκιμών μονάδας μπορεί να αφήσει αδιάγνωστα σφάλματα σε μεμονωμένα συστατικά του λογισμικού, τα οποία ενδέχεται να εκδηλωθούν σε μεταγενέστερα στάδια του κύκλου ζωής του προϊόντος, αυξάνοντας το κόστος επιδιόρθωσης και συντήρησης. Επιπροσθέτως, οι ελλείψεις στην αυτοματοποίηση των δοκιμών αποτελούν έναν ακόμη σημαντικό παράγοντα συσσώρευσης τεχνικού χρέους. Η αποτυχία αυτοματοποίησης των διαδικασιών ελέγχου οδηγεί συχνά σε χρονοβόρες και επιρρεπείς σε σφάλματα διαδικασίες χειροκίνητης δοκιμής. Αυτό όχι μόνο αυξάνει τον απαιτούμενο χρόνο και κόστος για τη διεξαγωγή των δοκιμών, αλλά επίσης μειώνει την αποτελεσματικότητα και την αξιοπιστία της διαδικασίας ελέγχου. Για παράδειγμα, η έλλειψη αυτοματοποιημένων δοκιμών παλινδρόμησης μπορεί να καταστήσει δύσκολο τον εντοπισμό νέων σφαλμάτων που εισάγονται κατά την ανάπτυξη νέων χαρακτηριστικών ή την επιδιόρθωση υφιστάμενων προβλημάτων.

Η φάση συντήρησης και υποστήριξης αποτελεί το τελικό και συχνά μακροβιότερο στάδιο στον κύκλο ζωής του λογισμικού. Ξεκινά μετά την επιτυχή ανάπτυξη και εγκατάσταση του συστήματος

και συνεχίζεται καθ' όλη τη διάρκεια της λειτουργικής ζωής του. Ο πρωταρχικός στόχος αυτής της φάσης είναι η διασφάλιση της συνεχούς λειτουργικότητας, αποδοτικότητας και ασφάλειας του συστήματος, καθώς και η προσαρμογή του στις μεταβαλλόμενες ανάγκες των χρηστών και του επιχειρησιακού περιβάλλοντος. Ωστόσο, η φάση συντήρησης και υποστήριξης μπορεί επίσης να συμβάλει στη συσσώρευση τεχνικού χρέους. Ένας σημαντικός παράγοντας είναι η ανεπαρκής τεκμηρίωση. Η έλλειψη επαρκούς και επικαιροποιημένης τεκμηρίωσης μπορεί να δυσχεράνει τη διαδικασία συντήρησης και αναβάθμισης του συστήματος. Για παράδειγμα, η απουσία λεπτομερών διαγραμμάτων αρχιτεκτονικής ή η ελλιπής τεκμηρίωση του κώδικα μπορεί να καθυστερήσει σημαντικά την εφαρμογή αλλαγών ή την επίλυση προβλημάτων, αυξάνοντας το κόστος συντήρησης. Επιπλέον, οι καθυστερημένες ενημερώσεις ασφαλείας αποτελούν άλλη μια πηγή τεχνικού χρέους σε αυτή τη φάση. Η παράλειψη ή καθυστέρηση εφαρμογής ενημερώσεων ασφαλείας μπορεί να εκθέσει το σύστημα σε κινδύνους και να απαιτήσει εκτεταμένες παρεμβάσεις στο μέλλον. Για παράδειγμα, η μη έγκαιρη αντιμετώπιση γνωστών ευπαθειών μπορεί να οδηγήσει σε παραβιάσεις ασφαλείας, απαιτώντας εκτεταμένες προσπάθειες αποκατάστασης και πιθανώς σημαντικές αλλαγές στην αρχιτεκτονική του συστήματος.

Η μακροπρόθεσμη αγνόηση του τεχνικού χρέους επιφέρει αναπόφευκτες και πολυδιάστατες συνέπειες στην ανάπτυξη, διαχείριση και λειτουργία του λογισμικού. Συγκεκριμένα, η συσσώρευση τεχνικού χρέους δύναται να οδηγήσει σε προοδευτική φθορά του λογισμικού, αυξημένη πολυπλοκότητα στη διαχείριση του κώδικα και αυξημένο κίνδυνο αποτυχίας του έργου. Οι επιπτώσεις αυτές εκδηλώνονται μέσω διαφόρων παραμέτρων, συμπεριλαμβανομένης της αύξησης του κόστους συντήρησης, της μείωσης της ευελιξίας στην ανάπτυξη νέων λειτουργιών, της αυξημένης πιθανότητας εμφάνισης σφαλμάτων και ελαττωμάτων, της μειωμένης παραγωγικότητας των προγραμματιστών και των περιορισμών στην απόδοση και κλιμάκωση του συστήματος. Συνεπώς, η συστηματική διαχείριση και ελαχιστοποίηση του τεχνικού χρέους καθίσταται κρίσιμη για τη διασφάλιση της μακροπρόθεσμης βιωσιμότητας και αποτελεσματικότητας των έργων λογισμικού.

2.1.2. Software Traceability

Η ιχνηλασιμότητα λογισμικού είναι η διαδικασία περιγραφής και παρακολούθησης της ζωής ενός τεχνουργήματος λογισμικού που παράγεται από τις απαιτήσεις καθ' όλη τη διάρκεια της φάσης της ανάπτυξης. Συγκεκριμένα, συνεπάγεται την ικανότητα καθιέρωσης σχέσεων ένα προς ένα μεταξύ ενός ευρέος φάσματος τεχνουργημάτων λογισμικού και της ενσωμάτωσής τους. Η ιχνηλασιμότητα στη μηχανική λογισμικού αντιπροσωπεύει μια τεχνική που χρησιμοποιείται για τη δημιουργία μιας καλά διαμορφωμένης σύνδεσης μεταξύ των εκδόσεων λογισμικού και των εξαρτήσεων που υπάρχουν εντός του λογισμικού.

Η ιχνηλασιμότητα λογισμικού παίζει σημαντικό ρόλο στην ανάπτυξη λογισμικού σήμερα λόγω της μεγάλης κλίμακας των σύγχρονων έργων λογισμικού, της πολυπλοκότητας που συνεχώς αυξάνεται και του αριθμού των προγραμματιστών που είναι υπεύθυνοι για το έργο. Όλοι αυτοί οι παράγοντες πρέπει να διαχειριστούν και να περιοριστούν για να ενισχυθεί η παράδοση και η συντήρηση του λογισμικού. Η ιχνηλασιμότητα λογισμικού επιλύει οποιοδήποτε ζήτημα προκύπτει παρέχοντας τα εξής:

- **Διασφάλιση Συμμόρφωσης και Λογοδοσίας:** Σε πολλούς κλάδους, η κανονιστική συμμόρφωση αποτελεί κρίσιμη απαίτηση. Η ιχνηλασιμότητα λογισμικού συμβάλλει στη διασφάλιση της τήρησης όλων των κανονιστικών προτύπων και απαιτήσεων συμμόρφωσης, παρέχοντας μια σαφή διαδρομή ελέγχου για όλα τα τεχνουργήματα του έργου. Αυτή η ιχνηλασιμότητα εξασφαλίζει ότι κάθε απαίτηση έχει εκπληρωθεί, κάθε αλλαγή έχει τεκμηριωθεί και όλες οι αποφάσεις μπορούν να δικαιολογηθούν. Αυτό είναι ιδιαίτερα σημαντικό σε τομείς όπως η υγειονομική περίθαλψη, τα χρηματοοικονομικά και η αεροδιαστημική, όπου η τήρηση των προτύπων είναι υποχρεωτική.
- **Ενίσχυση Ποιότητας και Μείωση Σφαλμάτων:** Η ιχνηλασιμότητα συμβάλλει στη διατήρηση υψηλής ποιότητας λογισμικού διασφαλίζοντας ότι όλες οι απαιτήσεις υλοποιούνται με ακρίβεια και πληρότητα. Ιχνηλατώντας κάθε απαίτηση στον αντίστοιχο κώδικα, δοκιμές και έγγραφα σχεδιασμού, οι ομάδες μπορούν να επαληθεύσουν ότι έχουν αντιμετωπιστεί όλες οι πτυχές της απαίτησης. Αυτό μειώνει τον κίνδυνο σφαλμάτων και παραλείψεων, οδηγώντας σε ένα πιο αξιόπιστο και ισχυρό προϊόν λογισμικού.

- **Διευκόλυνση Ανάλυσης Επιπτώσεων:** Όταν προτείνονται αλλαγές, οι χάρτες ιχνηλασιμότητας επιτρέπουν στις ομάδες να αξιολογήσουν γρήγορα τον αντίκτυπό τους. Κατανοώντας τις σχέσεις μεταξύ διαφόρων τεχνουργημάτων, οι ομάδες μπορούν να εντοπίσουν ποια μέρη του συστήματος θα επηρεαστούν και να διασφαλίσουν ότι γίνονται όλες οι απαραίτητες προσαρμογές. Αυτό μειώνει τον κίνδυνο ακούσιων συνεπειών και βοηθά στην αποτελεσματικότερη διαχείριση των αλλαγών.
- **Βελτίωση Διαχείρισης Έργου και Συντονισμού:** Η ανάπτυξη λογισμικού συχνά περιλαμβάνει πολλαπλές ομάδες που εργάζονται σε διαφορετικά στοιχεία του έργου. Η ιχνηλασιμότητα παρέχει μια ενοποιημένη εικόνα του πώς όλα αυτά τα στοιχεία συνδυάζονται, γεγονός που βοηθά στον συντονισμό και την επικοινωνία. Οι διαχειριστές έργων μπορούν να χρησιμοποιούν δεδομένα ιχνηλασιμότητας για να παρακολουθούν την πρόοδο, να διασφαλίζουν ότι όλα τα μέρη του έργου είναι ευθυγραμμισμένα με τις αρχικές απαιτήσεις και να λαμβάνουν τεκμηριωμένες αποφάσεις σχετικά με την κατανομή πόρων και τα χρονοδιαγράμματα.
- **Διευκόλυνση Συντήρησης και Εξέλιξης:** Τα συστήματα λογισμικού πρέπει να εξελίσσονται με την πάροδο του χρόνου για να ανταποκρίνονται στις μεταβαλλόμενες απαιτήσεις και τις τεχνολογικές εξελίξεις. Η ιχνηλασιμότητα είναι καθοριστική κατά τις φάσεις συντήρησης και εξέλιξης, καθώς βοηθά τους προγραμματιστές να κατανοήσουν τις αρχικές αποφάσεις σχεδιασμού και τις σχέσεις μεταξύ διαφορετικών μερών του συστήματος. Αυτή η γνώση είναι απαραίτητη για την πραγματοποίηση ακριβών και αποτελεσματικών αλλαγών χωρίς την εισαγωγή νέων σφαλμάτων ή την παλινδρόμηση λειτουργικότητας.
- **Ενίσχυση Κατανόησης Προγράμματος:** Οι χάρτες ιχνηλασιμότητας διευκολύνουν τα νέα μέλη της ομάδας να κατανοήσουν τη δομή και τη λειτουργικότητα του συστήματος. Παρέχοντας μια οπτική αναπαράσταση του τρόπου σύνδεσης διαφορετικών τεχνουργημάτων, η ιχνηλασιμότητα βοηθά τους νεοεισερχόμενους να ενημερωθούν γρήγορα, μειώνοντας την καμπύλη εκμάθησης και βελτιώνοντας τη συνολική παραγωγικότητα.
- **Υποστήριξη Επαναχρησιμοποίησης και Επανασχεδιασμού:** Οι πληροφορίες ιχνηλασιμότητας μπορεί να είναι ανεκτίμητες κατά την επαναχρησιμοποίηση ή τον επανασχεδιασμό υφιστάμενων στοιχείων λογισμικού. Κατανοώντας πώς σχεδιάστηκαν και

υλοποιήθηκαν αρχικά αυτά τα στοιχεία, οι ομάδες μπορούν να τα ενσωματώσουν πιο αποτελεσματικά σε νέα έργα ή να τα επανασχεδιάσουν για να ανταποκριθούν σε νέες απαιτήσεις. Αυτό υποστηρίζει την αποδοτικότερη επαναχρησιμοποίηση των περιουσιακών στοιχείων λογισμικού και βοηθά στη διατήρηση της αξίας των προηγούμενων προσπαθειών ανάπτυξης.

- **Διαχείριση Τεχνικού Χρέους:** Το τεχνικό χρέος, η συσσώρευση παρακάμψεων και μη βέλτιστων λύσεων, μπορεί να εμποδίσει τη συντήρηση και την εξέλιξη του λογισμικού εάν δεν διαχειρίζεται σωστά. Οι χάρτες ιχνηλασιμότητας που επισημαίνουν συγκεκριμένα περιοχές τεχνικού χρέους επιτρέπουν στις ομάδες να αντιμετωπίσουν προληπτικά αυτά τα ζητήματα. Δίνοντας προτεραιότητα και συστηματικά μειώνοντας το τεχνικό χρέος, οι ομάδες μπορούν να βελτιώσουν τη μακροπρόθεσμη υγεία και βιωσιμότητα των συστημάτων λογισμικού τους.

2.1.3. Software Traceability Maps

Για τη βελτιστοποίηση της ιχνηλασιμότητας και την αξιοποίηση των πλεονεκτημάτων της, θα πρέπει να αναπτυχθεί ένα εργαλείο που καλύπτει όλα τα διαφορετικά ζητήματα και είναι εύκολο για όλους να το κατανοήσουν και να το χρησιμοποιήσουν. Το όνομα αυτού του εργαλείου είναι χάρτες ιχνηλασιμότητας. Οι χάρτες ιχνηλασιμότητας είναι οπτικές αναπαραστάσεις των σχέσεων μεταξύ διαφορετικών τεχνουργημάτων σε ένα έργο λογισμικού. Αυτά τα τεχνουργήματα μπορεί να περιλαμβάνουν απαιτήσεις, έγγραφα σχεδιασμού, πηγαίο κώδικα, περιπτώσεις δοκιμών και άλλα σχετικά στοιχεία του έργου. Ο κύριος σκοπός των χαρτών ιχνηλασιμότητας είναι να παρέχουν μια σαφή και διαισθητική επισκόπηση του τρόπου με τον οποίο αυτά τα στοιχεία διασυνδέονται, διασφαλίζοντας ότι όλες οι απαιτήσεις υλοποιούνται και επικυρώνονται σωστά σε όλο τον κύκλο ζωής της ανάπτυξης λογισμικού.

Οι χάρτες ιχνηλασιμότητας έχουν αναβαθμίσει την ιχνηλασιμότητα λογισμικού τόσο για τους προγραμματιστές όσο και για τους ενδιαφερόμενους με τους εξής τρόπους:

- **Ενισχυμένη Κατανόηση:** Παρέχοντας μια οπτική αναπαράσταση των τεχνουργημάτων του έργου και των σχέσεών τους, οι χάρτες ιχνηλασιμότητας διευκολύνουν τα μέλη της ομάδας να κατανοήσουν τη συνολική δομή και ροή του έργου.

- Βελτιωμένη Διασφάλιση Ποιότητας: Οι χάρτες ιχνηλασιμότητας βοηθούν στη διασφάλιση ότι όλες οι απαιτήσεις αντιμετωπίζονται στις φάσεις σχεδιασμού και υλοποίησης και ότι όλες οι υλοποιημένες λειτουργίες δοκιμάζονται επαρκώς.
- Ανάλυση Επιπτώσεων: Όταν προτείνονται αλλαγές, οι χάρτες ιχνηλασιμότητας μπορούν γρήγορα να δείξουν ποια τεχνουργήματα θα επηρεαστούν, βοηθώντας τις ομάδες να αξιολογήσουν τον αντίκτυπο των αλλαγών και να σχεδιάσουν ανάλογα.
- Συντήρηση και Εξέλιξη: Οι χάρτες ιχνηλασιμότητας είναι ανεκτίμητοι κατά τη συντήρηση και την εξέλιξη, παρέχοντας ένα σαφές αρχείο για το πώς κατασκευάστηκε το σύστημα και πώς διασυνδέονται τα διάφορα μέρη του.
- Κανονιστική Συμμόρφωση: Για έργα σε ρυθμιζόμενους κλάδους, οι χάρτες ιχνηλασιμότητας παρέχουν την απαραίτητη τεκμηρίωση για να αποδειχθεί ότι όλες οι απαιτήσεις έχουν εκπληρωθεί και όλες οι αλλαγές έχουν παρακολουθηθεί σωστά.

Τέλος, οι χάρτες ιχνηλασιμότητας μπορούν συγκεκριμένα να αντιμετωπίσουν ζητήματα διαχείρισης τεχνικού χρέους χαρτογραφώντας περιοχές του λογισμικού που έχουν συσσωρεύσει τεχνικό χρέος, επισημαίνοντας τις συνδέσεις τους με άλλα μέρη του συστήματος. Η σημασία των χαρτών ιχνηλασιμότητας τεχνικού χρέους έγκειται στην ικανότητά τους να:

- Εντοπίζουν και Ιεραρχούν το Χρέος: Οπτικοποιώντας το τεχνικό χρέος, αυτοί οι χάρτες βοηθούν τις ομάδες να εντοπίσουν κρίσιμες περιοχές που απαιτούν άμεση προσοχή. Μπορούν να ιεραρχήσουν τις προσπάθειες αποκατάστασης του χρέους με βάση τον αντίκτυπο στο συνολικό σύστημα.
- Βελτιώνουν τη Λήψη Αποφάσεων: Με μια σαφή εικόνα του πώς κατανέμεται το τεχνικό χρέος και των πιθανών αλυσιδωτών επιπτώσεών του, οι διαχειριστές έργων και οι προγραμματιστές μπορούν να λαμβάνουν πιο τεκμηριωμένες αποφάσεις σχετικά με το πότε και πού να αντιμετωπίσουν το τεχνικό χρέος, εξισορροπώντας τους βραχυπρόθεσμους στόχους του έργου με τη μακροπρόθεσμη συντηρησιμότητα.
- Ενισχύουν την Επικοινωνία: Αυτοί οι χάρτες λειτουργούν ως ένα αποτελεσματικό εργαλείο επικοινωνίας μεταξύ των ενδιαφερομένων, παρέχοντας μια κοινή κατανόηση του τοπίου του τεχνικού χρέους. Αυτή η κοινή προοπτική είναι κρίσιμη για την ευθυγράμμιση των προτεραιοτήτων και των στρατηγικών μεταξύ διαφορετικών ομάδων.

- Διευκολύνουν τη Συνεχή Βελτίωση: Ενημερώνοντας και αναφέροντας συνεχώς τους χάρτες ιχνηλασιμότητας τεχνικού χρέους, οι ομάδες μπορούν να ενσωματώσουν τη διαχείριση χρέους στις τακτικές ροές εργασίας τους, καλλιεργώντας μια κουλτούρα συνεχούς βελτίωσης και προληπτικής διαχείρισης χρέους.

2.2. Σχετική έρευνα

Η διαχείριση του τεχνικού χρέους αποτελεί έναν σημαντικό τομέα έρευνας και πρακτικής στην ανάπτυξη λογισμικού, με στόχο την αναγνώριση, την παρακολούθηση και τη μείωση των επιπτώσεων του τεχνικού χρέους στη συντηρησιμότητα και την ποιότητα του λογισμικού. Στην βιβλιογραφία έχουν παρουσιαστεί διάφορες μεθοδολογίες διαχείρισης του τεχνικού χρέους σε διαφορετικά πλαίσια ανάπτυξης. Για την διαχείριση του τεχνικού χρέους αναπτύχθηκε η μέθοδος SQALE, η οποία παρέχει μια δομημένη προσέγγιση για την ανάλυση και την παρακολούθηση του τεχνικού χρέους, βοηθώντας τις ομάδες ανάπτυξης να εντοπίζουν και να αντιμετωπίζουν προβλήματα χρέους νωρίς στη διαδικασία ανάπτυξης. Επιπλέον, έχει γίνει διερευνύση των στοιχείων που απαιτούνται για τη διαχείριση του τεχνικού χρέους μέσω μιας συστηματικής χαρτογράφησης, όπου προσδιορίζονται οι μέθοδοι οπτικοποίησης και τα εργαλεία που είναι απαραίτητα για την αποτελεσματική παρακολούθηση και διαχείριση του τεχνικού χρέους. Για τις μεθόδους οπτικοποίησης έχουν μελετηθεί αναλυτικά οι χάρτες ιχνηλασιμότητας, στους οποίους βασίστηκε μια μέθοδος για την υποστήριξη της εξέλιξης των εννοιολογικών μοντέλων. Αυτή η μέθοδος συνδυάζει την ιχνηλασιμότητα με την ανάλυση και την παρακολούθηση των αλλαγών στα μοντέλα λογισμικού, καθιστώντας την ιδανική για τη διαχείριση του τεχνικού χρέους σε συστήματα που απαιτούν συνεχή ενημέρωση και προσαρμογή. Τέλος, έχουν παραχθεί μελέτες που παρέχουν μια εκτενή επισκόπηση των μεθόδων και των εργαλείων που χρησιμοποιούνται για την αναγνώριση και τη διαχείριση του τεχνικού χρέους, επισημαίνοντας τη σημασία των χαρτών ιχνηλασιμότητας για την οπτικοποίηση και την παρακολούθηση της εξέλιξης του χρέους σε διάφορα επίπεδα του λογισμικού. Το ερευνητικό, αυτό, ενδιαφέρον υποδεικνύει την πολυδιάστατη φύση του τεχνικού χρέους και τη σημασία των χαρτών ιχνηλασιμότητας στη διαχείρισή του. Η ανάπτυξη του λογισμικού συστήματος της παρούσας εργασίας, το οποίο εντοπίζει το τεχνικό χρέος και οπτικοποιεί την εξέλιξή του με τη βοήθεια των χαρτών ιχνηλασιμότητας, έρχεται να συμπληρώσει την υπάρχουσα έρευνα, προσφέροντας ένα πρακτικό εργαλείο για την αποτελεσματική διαχείριση του τεχνικού χρέους σε πραγματικό χρόνο.

3. Μεθοδολογίες και Εργαλεία

3.1. Java



Figure 1: Java programming language logo

Η Java είναι μια υψηλού επιπέδου αντικειμενοστρεφής γλώσσα προγραμματισμού η οποία αναπτύχθηκε από τον James Gosling στην εταιρεία Sun Microsystems. Η κυκλοφορία της έγινε τον Μάιο του 1995, μέσω της Java πλατφόρμας της εταιρείας αποτελώντας ένα βασικό στοιχείο αυτής. Σχεδιάστηκε και αναπτύχθηκε ως μια γενικού σκοπού γλώσσα προγραμματισμού η οποία παρέχει στους προγραμματιστές την δυνατότητα να παράγουν εφαρμογές οι οποίες θα ήταν συμβατές με οποιαδήποτε πλατφόρμα. Οι εφαρμογές της Java μεταγλωττίζονται σε byte-κώδικα ο οποίος μπορεί να εκτελεστεί από οποιοδήποτε Java Virtual Machine (JVM), ανεξάρτητα από την υποκείμενη αρχιτεκτονική του εκάστοτε υπολογιστή.

Όπως αναφέρθηκε η Java είναι αντικειμενοστρεφής γλώσσα προγραμματισμού, αυτό σημαίνει ότι ακολουθεί τους κανόνες και την συμπεριφορά του αντικειμενοστρεφούς προγραμματισμού. Ο αντικειμενοστρεφής προγραμματισμός είναι ένα είδος προγραμματισμού στο οποίο ένα σύστημα αποτελείται από τις οντότητες και από τις συσχετίσεις μεταξύ των οντοτήτων. Οι οντότητες αυτές ονομάζονται αντικείμενα και παρέχουν λειτουργικότητα και δεδομένα στο σύστημα. Το αντικείμενο, στην πράξη, είναι ένας αφηρημένος τύπος δεδομένων που περιέχει κατάλληλα πεδία και μεθόδους με τα οποία αλληλεπιδρά με τα υπόλοιπα αντικείμενα του συστήματος. Οι συσχετίσεις καθώς και η αλληλεπίδραση των αντικειμένων πραγματοποιούνται με βάση τις

θεμελιώδεις αρχές του αντικειμενοστρεφή προγραμματισμού (την κληρονομικότητα, τον πολυμορφισμό, την ενθυλάκωση και την αφαίρεση).

Structure of Object-Oriented Programming



Figure 2: Structure of Object-Oriented Programming

Το Java Virtual Machine (JVM) είναι μια εικονική μηχανή που επιτρέπει σε έναν υπολογιστή να εκτελέσει προγράμματα που αναπτύχθηκαν σε Java, όπως και προγράμματα που αναπτύχθηκαν σε κάποια άλλη γλώσσα προγραμματισμού τα οποία επίσης μεταγλωττίζονται σε byte-κώδικα της Java. Το JVM είναι μέρος του περιβάλλοντος εκτέλεσης της γλώσσα Java. Περιγράφεται λεπτομερώς από μια προδιαγραφή που περιέχει την επίσημη περιγραφή των απαιτήσεων μιας υλοποίησης του JVM.

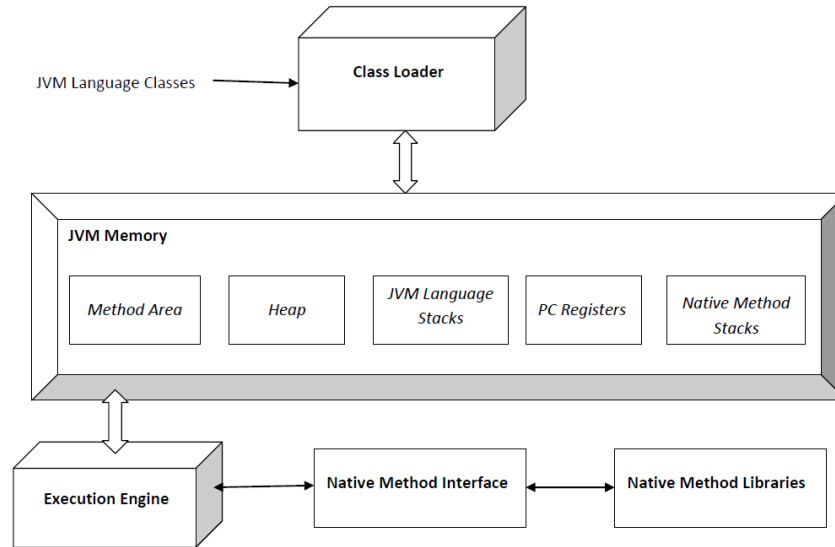


Figure 3: Java Virtual Machine architecture

Η αρχιτεκτονική αυτή έδωσε στην Java την φορητότητα και την απόδοση που χρειαζόνταν οι τότε εφαρμογές και έτσι από την αρχή πήρε μεγάλο προβάδισμα κατακτώντας ένα μεγάλο μέρος της κοινότητας των προγραμματιστών. Από το 2001 μέχρι και το 2019 η Java κατείχε την πρώτη θέση σε δημοτικότητα για το μεγαλύτερο χρονικό διάστημα, μεταξύ των υπόλοιπων γλωσσών προγραμματισμού. Τα τελευταία 5 χρόνια, έχει παρατηρηθεί μια μείωση της δημοτικότητάς της, καθώς νεότερες γλώσσες βασισμένες στο JVM κατακτούν τις πρώτες θέσεις.



Figure 4: Java programming language popularity chart

3.2. Spring Boot



Figure 5: Spring Boot logo

Το Spring Boot είναι ένα Java framework ανοικτού κώδικα και χρησιμοποιείται για τον προγραμματισμό αυτόνομων εφαρμογών. Το Spring Boot είναι μια επέκταση convention-over-configuration για την πλατφόρμα Spring της Java, η οποία προορίζεται να βοηθήσει στην ελαχιστοποίηση των προβλημάτων διαμόρφωσης κατά τη δημιουργία εφαρμογών. Το μεγαλύτερο μέρος της εφαρμογής μπορεί να διαμορφωθεί εκ των προτέρων χρησιμοποιώντας την πλατφόρμα που προσφέρεται από την ομάδα Spring για την διαμόρφωση της εφαρμογής καθώς και των βιβλιοθηκών τρίτων που προσφέρονται. Η χρήση αυτού του framework συνήθως περιλαμβάνει την δημιουργία μικροϋπηρεσιών, εφαρμογών διαδικτύου αλλά και πολλά διαφορετικά έργα Java λόγω της ευκολίας που προσφέρει στην χρήση και την διαμόρφωση του.

3.3. Neo4j



Figure 6: Neo4j logo

Το Neo4j είναι ένα σύστημα διαχείρισης βάσεων δεδομένων γράφων (GDBMS) που αναπτύχθηκε από τη Neo4j, Inc. το 2017. Τα στοιχεία δεδομένων που αποθηκεύει το Neo4j είναι οι κόμβοι και οι ακμές που τους συνδέουν όπως επίσης και τα χαρακτηριστικά τους, είτε των κόμβων είτε των ακμών. Είναι μια βάση δεδομένων συναλλαγών συμβατή με ACID, με εγγενή αποθήκευση και επεξεργασία γράφων που υλοποιείται σε Java και είναι προσβάσιμο από οποιοδήποτε λογισμικό

γραμμένο σε άλλες γλώσσες χρησιμοποιώντας τη γλώσσα ερωτημάτων Cypher μέσω ενός συναλλακτικού HTTP endpoint, ή μέσω του δυαδικού πρωτοκόλλου Bolt. Το Neo4j έχει πολλά προτερήματα, μεταξύ άλλων προσφέρει:

- Μια cloud-ready αρχιτεκτονική που κλιμακώνεται ανάλογα με τις ανάγκες των δεδομένων και ελαχιστοποιεί το κόστος υποδομής, ενώ μεγιστοποιεί την απόδοση σε συνδεδεμένα σύνολα δεδομένων.
- Υψηλής απόδοσης συναλλακτικά και αναλυτικά ερωτήματα. Τα ερωτήματα πολλαπλών βημάτων εκτελούνται ομαλά με τον βέλτιστο σχεδιασμό ερωτημάτων και το Cypher Parallel Runtime επιτρέπει εξαιρετικά βελτιωμένη εκτέλεση αναλυτικών ερωτημάτων σε μεγάλα τμήματα του γράφου.

Τέλος, το Neo4j έχει λάβει μεγάλη δημοτικότητα τα τελευταία χρόνια λόγω της ευχρηστίας, της απόδοσης και της κλιμάκωσης που προσφέρει το σύστημα αλλά και λόγω των αναγκών που έχουν προκύψει για βάσεις δεδομένων βασισμένες σε γράφους.

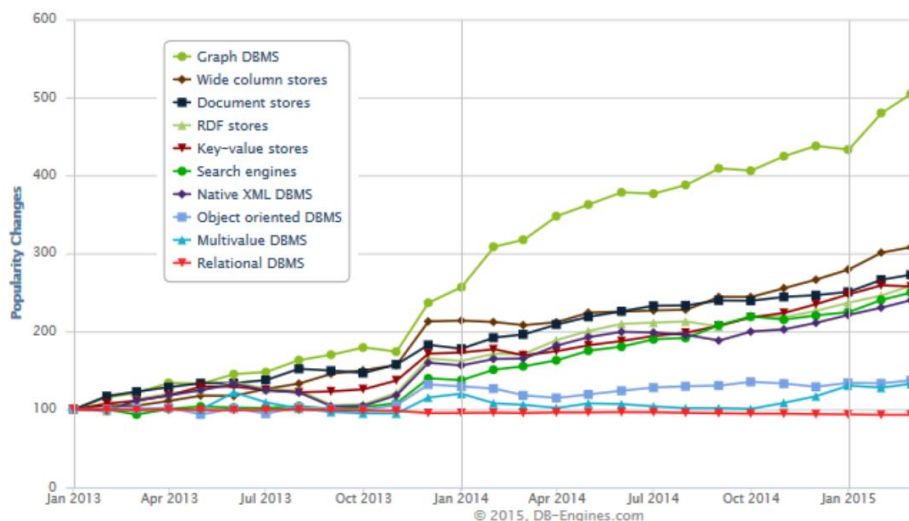


Figure 7: DBMS popularity chart

3.4. PostgreSQL



Figure 8: PostgreSQL logo

Η PostgreSQL είναι ένα ελεύθερο και ανοικτού κώδικα σύστημα διαχείρισης αντικειμενο-σχεσιακών βάσεων δεδομένων (RDBMS) με έμφαση στην επεκτασιμότητα και στη συμμόρφωση με την SQL. Η PostgreSQL παρέχει συναλλαγές με ιδιότητες ACID από το 2001, αυτόματα ανανεώσιμες και υλοποιημένες προβολές, ξένα κλειδιά και αποθηκευμένες διαδικασίες. Υποστηρίζεται από όλα τα σημαντικά λειτουργικά συστήματα και διαχειρίζεται ένα εύρος σεναρίων εργασίας από μεμονωμένες μηχανές έως αποθήκες δεδομένων ή υπηρεσίες ιστού με πολλούς ταυτόχρονους χρήστες. Η αφετηρία της PostgreSQL χρονολογείται το 1986 ως μέρος του έργου POSTGRES στο Πανεπιστήμιο της Καλιφόρνιας στο Μπέρκλεϊ και έχει περισσότερα από 35 χρόνια ενεργής ανάπτυξης της βασικής πλατφόρμας. Η PostgreSQL Global Development Group επικεντρώνεται μόνο στην ανάπτυξη μιας μηχανής βάσεων δεδομένων και στενά συνδεδεμένων στοιχείων. Αυτός ο πυρήνας είναι αυτό που περιλαμβάνει την ίδια την PostgreSQL, αλλά υπάρχει μια εκτεταμένη κοινότητα προγραμματιστών και ένα οικοσύστημα που παρέχει άλλα σημαντικά σύνολα χαρακτηριστικών. Αυτά περιλαμβάνουν χαρακτηριστικά μηχανής βάσεων δεδομένων ειδικού σκοπού, όπως αυτά που απαιτούνται για την υποστήριξη μιας γεωχωρικής (PostGIS) ή χρονικής βάσης δεδομένων ή χαρακτηριστικά που μιμούνται άλλα προϊόντα βάσεων δεδομένων. Το σύστημα αυτό, έχει αποκτήσει ισχυρή φήμη για την αποδεδειγμένη αρχιτεκτονική της, την αξιοπιστία, την ακεραιότητα των δεδομένων, το ισχυρό σύνολο χαρακτηριστικών, την επεκτασιμότητα και την αφοσίωση της κοινότητας ανοικτού κώδικα που βρίσκεται πίσω από το λογισμικό για τη συνεχή παροχή αποδοτικών και καινοτόμων λύσεων.

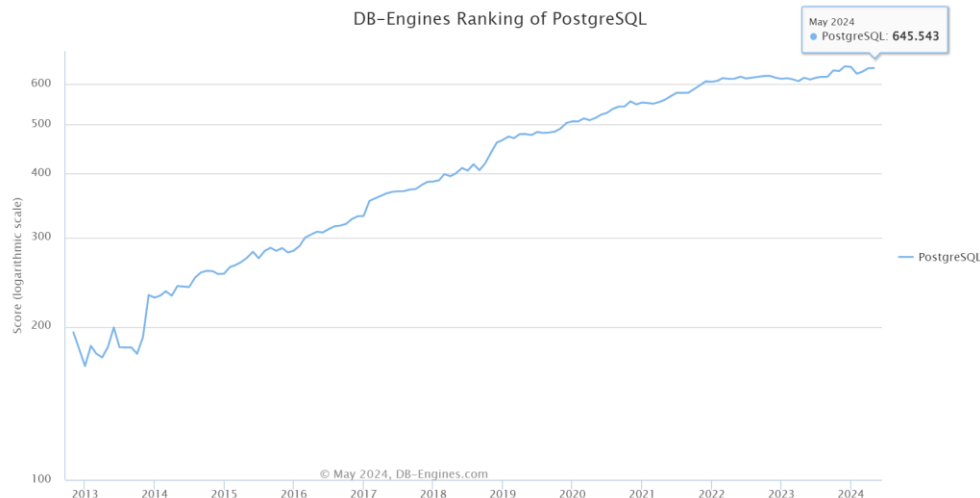


Figure 9: PostgreSQL ranking chart

3.5. JavaScript



Figure 10: JavaScript logo

Η JavaScript (JS) είναι μια διερμηνευμένη γλώσσα προγραμματισμού για ηλεκτρονικούς υπολογιστές που αποτέλεσε μέρος της υλοποίησης των φυλλομετρητών ιστού, ώστε τα σενάρια από την πλευρά του πελάτη (client-side scripts) να μπορούν να επικοινωνούν με τον χρήστη, να ανταλλάσσουν δεδομένα ασύγχρονα και να αλλάζουν δυναμικά το περιεχόμενο του εγγράφου που εμφανίζεται. Η γλώσσα προγραμματισμού JavaScript δημιουργήθηκε αρχικά από τον Brendan Eich της εταιρείας Netscape με την επωνυμία Mocha. Αργότερα, η Mocha μετονομάστηκε σε LiveScript, και τελικά σε JavaScript, κυρίως επειδή η ανάπτυξή της επηρεάστηκε περισσότερο από τη γλώσσα προγραμματισμού Java. Τον Νοέμβριο του 1996, η Netscape ανακοίνωσε ότι είχε υποβάλει τη γλώσσα JavaScript στο Ecma International (μια οργάνωση της τυποποίησης των

γλωσσών προγραμματισμού) για εξέταση ως βιομηχανικό πρότυπο, και στη συνέχεια το έργο είχε ως αποτέλεσμα την τυποποιημένη μορφή που ονομάζεται ECMAScript. Τον Ιανουάριο του 2009, το έργο CommonJS ιδρύθηκε με στόχο τον καθορισμό ενός κοινού προτύπου βιβλιοθήκης κυρίως για την ανάπτυξη της JavaScript έξω από το πρόγραμμα περιήγησης και μέσα σε άλλες τεχνολογίες (π.χ. server-side). Η γλώσσα απέκτησε μεγάλη επιτυχία ως γλώσσα στην πλευρά του πελάτη (client-side) για εκτέλεση κώδικα σε ιστοσελίδες, και περιλήφθηκε σε διάφορα προγράμματα περιήγησης στο Web. Η JavaScript είναι μια γλώσσα σεναρίων που βασίζεται στα πρωτότυπα (prototype-based), είναι δυναμική, με ασθενείς τύπους και έχει συναρτήσεις ως αντικείμενα πρώτης τάξης. Η JavaScript αντιγράφει πολλά ονόματα και συμβάσεις ονοματοδοσίας από τη Java, αλλά γενικά οι δύο αυτές γλώσσες δε σχετίζονται και έχουν πολύ διαφορετική σημασιολογία. Οι βασικές αρχές σχεδιασμού της JavaScript προέρχονται από τις γλώσσες προγραμματισμού Self και Scheme. Είναι γλώσσα βασισμένη σε διαφορετικά προγραμματιστικά παραδείγματα (multi-paradigm), υποστηρίζοντας αντικειμενοστρεφές, προστακτικό και συναρτησιακό στιλ προγραμματισμού. Γρήγορα έγινε δημοφιλής και σήμερα είναι μια από τις πιο διαδεδομένες γλώσσες προγραμματισμού στον κόσμο. Η δημοτικότητά της οφείλεται εν μέρει στην απλότητα και την ευελιξία της, καθώς και στην ικανότητά της να συνεργάζεται απρόσκοπτα με άλλες τεχνολογίες ιστού, όπως η HTML και η CSS. Με την άνοδο της ανάπτυξης εφαρμογών ιστού και κινητών τηλεφώνων, η JavaScript έχει γίνει απαραίτητη δεξιότητα για τους προγραμματιστές στη σημερινή τεχνολογική βιομηχανία. Εκτός από αυτό, οι ακόλουθοι είναι μερικοί από τους λόγους που καθιστούν τη JavaScript δημοφιλή.

1. Συμβατότητα με τον ιστό
2. Κυριαρχία του front-end
3. Ανάπτυξη από την πλευρά του διακομιστή με το Node.js
4. Μεγάλη κοινότητα προγραμματιστών
5. Ευελιξία σε διάφορους τομείς
6. Συνεχής βελτίωση
7. Εύκολη εκμάθηση
8. Απρόσκοπτη ενσωμάτωση με άλλες τεχνολογίες
9. Συμβατότητα με σύγχρονες πρακτικές ανάπτυξης
10. Βιομηχανικό πρότυπο για την ανάπτυξη ιστοσελίδων και πολλά άλλα.

3.6. TypeScript



Figure 11: TypeScript logo

Η TypeScript είναι μια ανοικτού κώδικα γλώσσα προγραμματισμού υψηλού επιπέδου που αναπτύχθηκε από τη Microsoft και προσθέτει στατική τυποποίηση με προαιρετικές επισημάνσεις τύπου στη JavaScript. Είναι σχεδιασμένη για την ανάπτυξη μεγάλων εφαρμογών και μεταγλωττίζεται σε JavaScript. Επειδή η TypeScript είναι ένα υπερσύνολο της JavaScript, όλα τα προγράμματα JavaScript είναι συντακτικά έγκυρα TypeScript, αλλά μπορεί να αποτύχουν στον έλεγχο τύπου για λόγους ασφαλείας. Η TypeScript κυκλοφόρησε στο κοινό τον Οκτώβριο του 2012, με την έκδοση 0.8, μετά από δύο χρόνια εσωτερικής ανάπτυξης στη Microsoft. Αμέσως μετά την αρχική δημόσια κυκλοφορία, ο Miguel de Icaza επαίνεσε την ίδια τη γλώσσα, αλλά επέκρινε την έλλειψη υποστήριξης IDE εκτός από το Microsoft Visual Studio, το οποίο δεν ήταν διαθέσιμο σε Linux και OS X εκείνη την εποχή. Από τον Απρίλιο του 2021 υπάρχει υποστήριξη σε αρκετά IDEs και επεξεργαστές κειμένου συμπεριλαμβανομένων των Emacs, Vim, WebStorm, Atom και του δικού της Microsoft Visual Studio Code. Το 2013 η TypeScript 0.9 πρόσθεσε υποστήριξη για generics. Από εκεί και έπειτα, η ανάπτυξη της γλώσσα εκτοξεύθηκε προσθέτοντας χρόνο με τον χρόνο περισσότερα χαρακτηριστικά και υποστήριξη στη γλώσσα. Η TypeScript μπορεί να χρησιμοποιηθεί για την ανάπτυξη εφαρμογών JavaScript τόσο για εκτέλεση από την πλευρά του πελάτη όσο και από την πλευρά του διακομιστή (όπως με το Node.js, το Deno ή το Bun). Για τη μεταγλώττιση είναι διαθέσιμες πολλαπλές επιλογές, όπως ο προεπιλεγμένος μεταγλωττιστής TypeScript ή ο μεταγλωττιστής Babel για τη μετατροπή της TypeScript σε JavaScript.

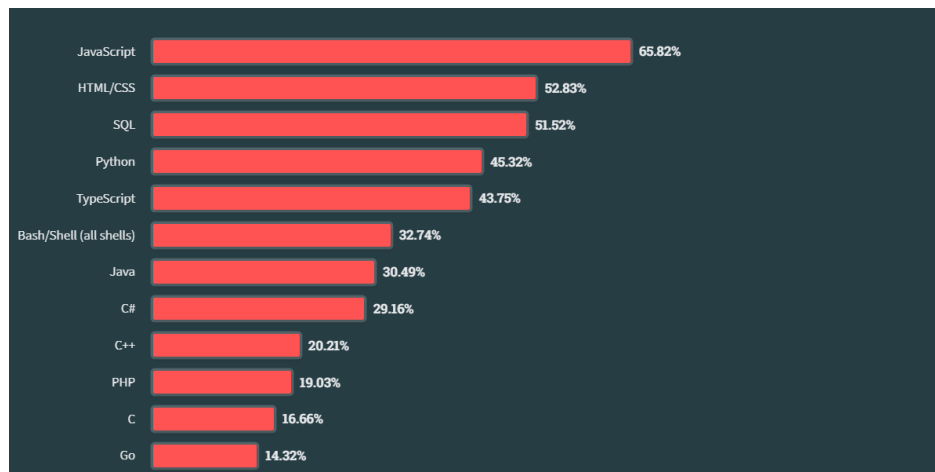


Figure 12: StackOverflow programming language ranking

3.7. React

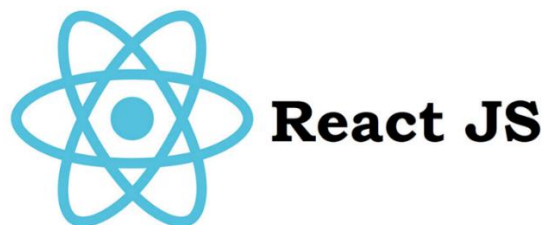


Figure 13: React JS logo

Η React είναι μια δωρεάν και ανοικτού κώδικα βιβλιοθήκη front-end JavaScript για τη δημιουργία διεπαφών χρήστη που βασίζονται σε στοιχεία. Συντηρείται από τη Meta (πρώην Facebook) και μια κοινότητα μεμονωμένων προγραμματιστών και εταιρειών. Η React δημιουργήθηκε από τον Jordan Walke, έναν μηχανικό λογισμικού της Meta, ο οποίος αρχικά ανέπτυξε ένα πρωτότυπο με την ονομασία "F-Bolt", προτού το μετονομάσει αργότερα σε "FaxJS". Χρησιμοποιήθηκε για πρώτη φορά στο News Feed του Facebook το 2011 και στη συνέχεια ενσωματώθηκε στο Instagram το 2012. Τον Μάιο του 2013, στο JSConf US, το έργο έγινε επίσημα open-sourced, σηματοδοτώντας ένα σημαντικό σημείο καμπής στην υιοθέτηση και την ανάπτυξή του. Στις 18 Απριλίου 2017, το Facebook ανακοίνωσε το React Fiber, ένα νέο σύνολο εσωτερικών αλγορίθμων για την απόδοση, σε αντίθεση με τον παλιό αλγόριθμο απόδοσης του React, το Stack. Το React

Fiber επρόκειτο να αποτελέσει τη βάση για κάθε μελλοντική βελτίωση και ανάπτυξη χαρακτηριστικών της βιβλιοθήκης React. Τα χαρακτηριστικά της React είναι τα εξής:

- Δηλωτική
- Συστατικά
- Συστατικά λειτουργιών
- React Hooks
- Εξαρτήματα διακομιστή
- Συστατικά κλάσης
- Δρομολόγηση
- Εικονικό DOM

Η React μπορεί να χρησιμοποιηθεί για την ανάπτυξη εφαρμογών μίας σελίδας, εφαρμογών για φορητές συσκευές ή εφαρμογών που παρουσιάζονται σε διακομιστή με πλαίσια όπως το Next.js. Επειδή το React ασχολείται μόνο με τη διεπαφή χρήστη και την απόδοση στοιχείων στο DOM (Document Object Model), οι εφαρμογές React συχνά βασίζονται σε βιβλιοθήκες για δρομολόγηση και άλλες λειτουργίες στην πλευρά του πελάτη. Ένα βασικό πλεονέκτημα του React είναι ότι επανεκκινεί μόνο τα τμήματα της σελίδας που έχουν αλλάξει, αποφεύγοντας την περιττή επανεκτέλεση αμετάβλητων στοιχείων DOM.

3.8. Node.js



Figure 14: Node.js logo

Το Node.js είναι μια πλατφόρμα ανάπτυξης λογισμικού (κυρίως διακομιστών) χτισμένη σε περιβάλλον JavaScript. Στόχος του Node είναι να παρέχει έναν εύκολο τρόπο δημιουργίας

κλιμακωτών διαδικτυακών εφαρμογών. Σε αντίθεση από τα περισσότερα σύγχρονα περιβάλλοντα ανάπτυξης εφαρμογών δικτύων μία διεργασία node δε στηρίζεται στην πολυνηματικότητα αλλά σε ένα μοντέλο ασύγχρονης επικοινωνίας εισόδου/εξόδου. Το Node.js δημιουργήθηκε από τον Ryan Dahl το 2009. Η δημιουργία και η συντήρηση του έργου χορηγήθηκε από την εταιρία Joyent. Η ιδέα για την ανάπτυξη του Node προήλθε από την ανάγκη του Ryan Dahl να βρει τον πιο αποδοτικό τρόπο να ενημερώνει τον χρήστη σε πραγματικό χρόνο για την κατάσταση ενός αρχείου που ανέβαζε στο διαδίκτυο. Επίσης επηρεάστηκε από το Mongrel του Zed Shaw. Επιπροσθέτως μετά από αποτυχημένα έργα σε C, Lua, Haskell η κυκλοφορία της μηχανής V8 (V8 JavaScript Engine) της Google τον ώθησε να ασχοληθεί με την JavaScript. Το Node χαρακτηρίζεται από την έμφαση στην ασύγχρονη επικοινωνία μεταξύ των υπολογιστικών πόρων. Αυτό επιτυγχάνεται με τη χρήση συμβάντων (events) που προσφέρει η JavaScript και ονομάζονται callbacks.

3.9. SonarQube



Figure 15: SonarQube logo

Το SonarQube είναι μια πλατφόρμα ανοικτού κώδικα που αναπτύχθηκε για τη συνεχή επιθεώρηση της ποιότητας του κώδικα για την εκτέλεση αυτόματων αναθεωρήσεων με στατική ανάλυση του κώδικα για τον εντοπισμό σφαλμάτων και «οσμών» κώδικα σε 29 γλώσσες προγραμματισμού. Η υλοποίηση του βασίστηκε στη γλώσσα προγραμματισμού Java και η αρχική του κυκλοφορία ήταν το 2006 – 2007 από τη SonarSource. Το SonarQube προσφέρει αναφορές σχετικά με τον διπλότυπο κώδικα, τα πρότυπα κωδικοποίησης, την κάλυψη κώδικα, την πολυπλοκότητα κώδικα, τα σχόλια, τα σφάλματα και τις συστάσεις ασφαλείας. Παρέχει αυτοματοποιημένη ανάλυση και ενσωμάτωση με τα εργαλεία Maven, Ant, Gradle, MSBuild και εργαλεία συνεχούς ολοκλήρωσης.

3.10. Docker



Figure 16: Docker logo

Το Docker είναι ένα σύνολο προϊόντων πλατφόρμας ως υπηρεσία (PaaS) που χρησιμοποιούν εικονικοποίηση σε επίπεδο λειτουργικού συστήματος για την παροχή λογισμικού σε πακέτα που ονομάζονται container. Το λογισμικό που φιλοξενεί τα εμπορευματοκιβώτια ονομάζεται Docker Engine, κυκλοφόρησε για πρώτη φορά το 2013 και αναπτύσσεται από τη Docker, Inc. Το εργαλείο αυτό χρησιμοποιείται για την αυτοματοποίηση της ανάπτυξης εφαρμογών σε ελαφριά εμπορευματοκιβώτια, ώστε οι εφαρμογές να μπορούν να λειτουργούν αποτελεσματικά σε διαφορετικά περιβάλλοντα μεμονωμένα. Τα εμπορευματοκιβώτια είναι απομονωμένα το ένα από το άλλο και συσσωρεύουν το δικό τους λογισμικό, βιβλιοθήκες και αρχεία ρυθμίσεων. Μπορούν να επικοινωνούν μεταξύ τους μέσω καλά καθορισμένων καναλιών. Επειδή όλα τα εμπορευματοκιβώτια μοιράζονται τις υπηρεσίες ενός ενιαίου πυρήνα λειτουργικού συστήματος, χρησιμοποιούν λιγότερους πόρους από τις εικονικές μηχανές. Το Docker μπορεί να συσκευάσει μια εφαρμογή και τις εξαρτήσεις της σε ένα εικονικό δοχείο που μπορεί να εκτελεστεί σε οποιονδήποτε υπολογιστή Linux, Windows ή macOS. Αυτό δίνει τη δυνατότητα στην εφαρμογή να εκτελείται σε διάφορες τοποθεσίες, όπως σε εγκαταστάσεις, σε δημόσιο ή ιδιωτικό νέφος. Όταν εκτελείται σε Linux, το Docker χρησιμοποιεί τα χαρακτηριστικά απομόνωσης πόρων του πυρήνα Linux (όπως cgroups και kernel namespaces) και ένα σύστημα αρχείων με δυνατότητα ένωσης για να επιτρέπει στα κοντέινερ να εκτελούνται μέσα σε μία μόνο περίπτωση Linux, αποφεύγοντας τα γενικά έξοδα εκκίνησης και συντήρησης εικονικών μηχανών. Η υποστήριξη του πυρήνα Linux για namespaces κυρίως απομονώνει την άποψη μιας εφαρμογής για το λειτουργικό περιβάλλον, συμπεριλαμβανομένων των δέντρων διεργασιών, του δικτύου, των αναγνωριστικών χρηστών και των προσαρτημένων συστημάτων αρχείων, ενώ οι ομάδες cgroups του πυρήνα παρέχουν περιορισμό των πόρων για τη μνήμη και τη CPU.

3.11. Git



Figure 17: Git logo

Το Git είναι ένα ελεύθερο και ανοικτού κώδικα καταναμημένο σύστημα ελέγχου εκδόσεων (VCS) που παρακολουθεί τις εκδόσεις των αρχείων. Το Git δημιουργήθηκε για χρήση στην ανάπτυξη του πυρήνα του Linux από τον Linus Torvalds το 2005. Οι σχεδιαστικοί στόχοι του περιλαμβάνουν την ταχύτητα, την ακεραιότητα των δεδομένων και την υποστήριξη καταναμημένων συστημάτων, μη γραμμικών ροών εργασίας - χιλιάδες παράλληλοι κλάδοι που εκτελούνται σε διαφορετικούς υπολογιστές. Χρησιμοποιείται συχνά για τον έλεγχο του πηγαίου κώδικα από προγραμματιστές που αναπτύσσουν συνεργατικά λογισμικό. Όπως συμβαίνει με τα περισσότερα άλλα καταναμημένα συστήματα ελέγχου εκδόσεων, και σε αντίθεση με τα περισσότερα συστήματα πελάτη-εξυπηρετητή, το Git διατηρεί ένα τοπικό αντίγραφο ολόκληρου του αποθετηρίου με ιστορικό και δυνατότητες παρακολούθησης εκδόσεων, ανεξάρτητα από την πρόσβαση στο δίκτυο ή έναν κεντρικό διακομιστή. Ένα αποθετήριο αποθηκεύεται σε κάθε υπολογιστή σε έναν τυπικό κατάλογο με πρόσθετα, κρυφά αρχεία για να παρέχει δυνατότητες ελέγχου εκδόσεων. Το Git παρέχει δυνατότητες συγχρονισμού των αλλαγών μεταξύ αποθετηρίων που μοιράζονται το ιστορικό - αντιγράφονται (κλωνοποιούνται) το ένα από το άλλο. Το Git υποστηρίζει συγχρονισμό με ένα αποθετήριο σε απομακρυσμένους υπολογιστές, ώστε να επιτυγχάνεται απομακρυσμένη συνεργασία. Αν και όλα τα αποθετήρια - με το ίδιο ιστορικό - είναι ομότιμα, οι προγραμματιστές συνήθως χρησιμοποιούν έναν κεντρικό διακομιστή για να φιλοξενήσουν ένα αποθετήριο για να κρατήσουν ένα ολοκληρωμένο αντίγραφο. Σήμερα, το Git είναι το τυπικό VCS για τη μεγάλη πλειοψηφία των έργων λογισμικού. Είναι το πιο δημοφιλές καταναμημένο VCS, με σχεδόν το 95% των προγραμματιστών να το αναφέρουν ως το κύριο VCS τους από το 2022. Είναι το πιο ευρέως χρησιμοποιούμενο εργαλείο διαχείρισης πηγαίου κώδικα μεταξύ των επαγγελματιών προγραμματιστών.

4. Ανάπτυξη του εργαλείου

Το εργαλείο που αναπτύχθηκε στο πλαίσιο της συγκεκριμένης εργασίας είναι ένα σύγχρονο λογισμικό σύστημα που διευκολύνει τη διαχείριση και οπτικοποίηση του τεχνικού χρέους και γενικότερα την παρακολούθηση των μετρικών ενός έργου λογισμικού. Είναι μια Full Stack εφαρμογή που απαρτίζεται από το επίπεδο πελάτη, υπεύθυνο για την προβολή της διεπαφής και της διάδρασης με τον χρήστη, το επίπεδο του εξυπηρετητή, υπεύθυνο για τη διαχείριση και επεξεργασία των δεδομένων και την επιτυχή διεκπεραίωση των αιτημάτων του πελάτη, καθώς και το επίπεδο των δεδομένων, υπεύθυνο για την αποθήκευση και διανομή των δεδομένων στις διάφορες λειτουργικές μονάδες του συστήματος. Στην ενότητα αυτή, θα παρουσιαστούν αναλυτικά η αρχιτεκτονική του συστήματος, όπως επίσης και τα διαφορετικά επίπεδα και τεχνολογίες του εργαλείου.

4.1. Αρχιτεκτονική σχεδίαση

Η αρχιτεκτονική σχεδίαση του εργαλείου στηρίχθηκε στη μέθοδο αρχιτεκτονικής μικροϋπηρεσιών. Η αρχιτεκτονική μικροϋπηρεσιών είναι μια αρχιτεκτονική μέθοδος που βασίζεται σε μια σειρά ανεξάρτητα αναπτυσσόμενων υπηρεσιών. Αυτές οι υπηρεσίες έχουν τη δική τους επιχειρηματική λογική με ένα συγκεκριμένο στόχο. Όλες οι απαραίτητες ενέργειες για την ανάπτυξη ενός λογισμικού γίνονται εντός κάθε υπηρεσίας. Οι μικροϋπηρεσίες αποσυνδέουν σημαντικούς επιχειρηματικούς τομείς σε ξεχωριστά και ανεξάρτητα λογισμικά συστήματα. Οι μικροϋπηρεσίες δε μειώνουν την πολυπλοκότητα, αλλά κάνουν οποιαδήποτε πολυπλοκότητα ορατή και πιο διαχειρίσιμη, διαχωρίζοντας τα καθήκοντα σε μικρότερες διεργασίες που λειτουργούν ανεξάρτητα η μία από την άλλη και συμβάλλουν στο σύνολο. Η επικοινωνία μεταξύ των μικροϋπηρεσιών βασίζεται στην αρχιτεκτονική REST.

Το REST (Representational State Transfer) είναι ένα αρχιτεκτονικό στιλ για τον σχεδιασμό δικτυακών εφαρμογών. Περιγράφει ένα σύνολο περιορισμών και αρχών για τη δημιουργία δικτυακών υπηρεσιών. Οι RESTful υπηρεσίες, όπως συνήθως αποκαλούνται, ακολουθούν αυτούς τους περιορισμούς για να επιτρέψουν την αποτελεσματική και επεκτάσιμη επικοινωνία μεταξύ του πελάτη και του διακομιστή.

Η ανάπτυξη του συγκεκριμένου εργαλείου χωρίστηκε σε τρία επίπεδα:

1. Το επίπεδο του πελάτη
2. Το επίπεδο του εξυπηρετητή
3. Το επίπεδο των δεδομένων

Το κάθε επίπεδο αποτελεί και αποτελείται από μικροϋπηρεσίες. Το επίπεδο του πελάτη στηρίζεται στις τεχνολογίες Node.js και React. Οι τεχνολογίες αυτές είναι υπεύθυνες για την αναπαράσταση των δεδομένων στον χρήστη καθώς και για τη διάδρασή του με το υπόλοιπο σύστημα. Το επίπεδο του εξυπηρετητή στηρίζεται στην τεχνολογία Spring Boot καθώς και στην υπηρεσία SonarQube η οποία λειτουργεί μέσω docker container. Τέλος, το επίπεδο δεδομένων περιέχει τις βάσεις δεδομένων Neo4j και PostgreSQL των συστημάτων Spring Boot και SonarQube, αντίστοιχα. Τα συστήματα διαχείρισης των βάσεων δεδομένων λειτουργούν εξ' ολοκλήρου σε docker containers.

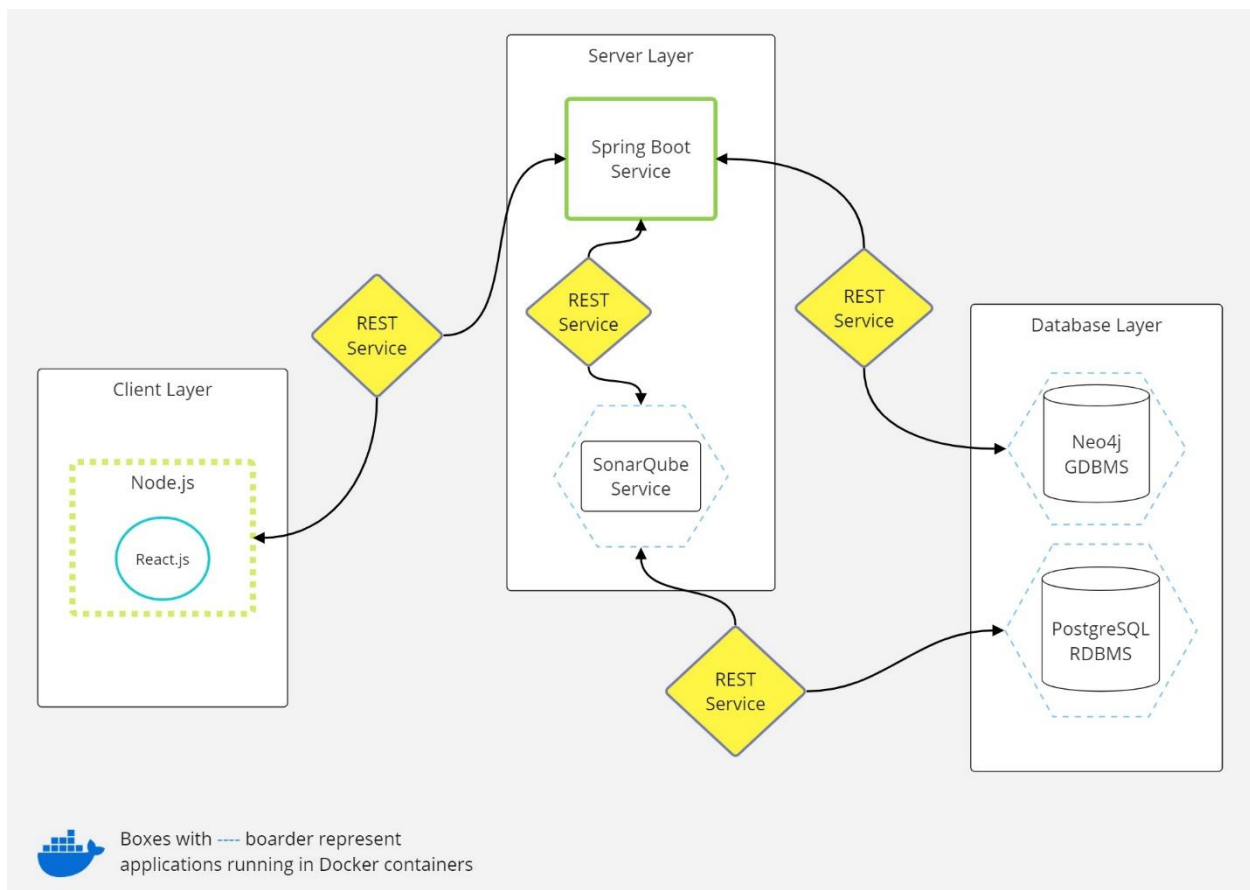


Figure 18: Tool architecture

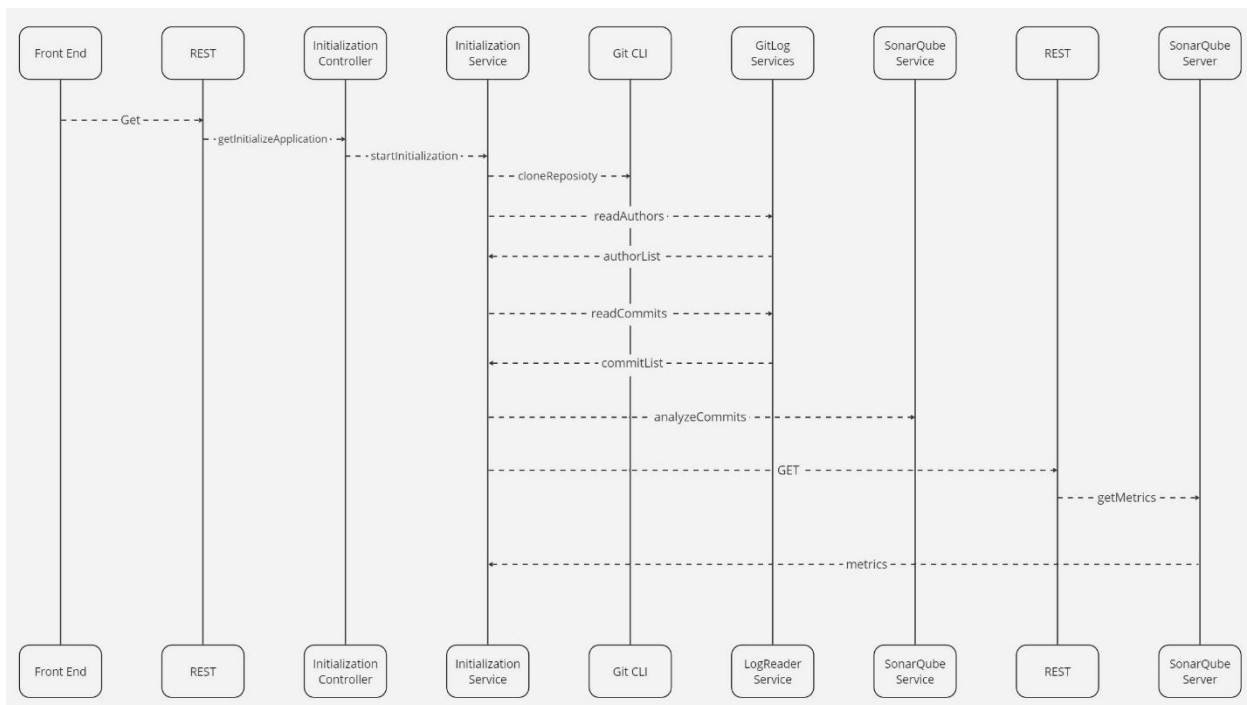


Figure 19: Project initialization Sequence Diagram

4.2. Ανάπτυξη Backend

Το Back End του εργαλείου αποτελείται από δύο επίπεδα, το επίπεδο του εξυπηρετητή και το επίπεδο των δεδομένων. Στο επίπεδο του εξυπηρετητή αναπτύχθηκε η εφαρμογή Java Spring Boot για την εξυπηρέτηση των εξερχόμενων αιτημάτων και χρησιμοποιήθηκε η υπηρεσία SonarQube, που βρίσκεται σε docker container, για την αποτελεσματική ανάλυση των έργων λογισμικού. Το επίπεδο των δεδομένων αποτελείται αποκλειστικά από υπηρεσίες, σε docker container, οι οποίες παρέχουν τις βάσεις δεδομένων Neo4j και PostgreSQL στο σύστημα. Η βάση δεδομένων Neo4j χρησιμοποιείται από τη Spring Boot εφαρμογή, ενώ η βάση δεδομένων PostgreSQL από την υπηρεσία SonarQube.

4.2.1. Server Layer

Η ανάπτυξη του Server Layer στηρίχθηκε στην τεχνολογία Spring Boot. Το Spring Boot προσφέρει ένα ολοκληρωμένο περιβάλλον ανάπτυξης για εφαρμογές εξυπηρετητών. Παρέχει πλήρη υποστήριξη αρχιτεκτονικής REST, διαχειρίζεται αποδοτικά πολλαπλά αιτήματα,

διευκολύνει την ανάπτυξη του κώδικα και επιτρέπει τη διασύνδεση με ποικίλα συστήματα μέσω των διαθέσιμων εξαρτήσεων. Η εφαρμογή του εξυπηρετητή χωρίζεται σε καταλόγους σύμφωνα με την υπηρεσία που παρέχει κάθε μονάδα του συστήματος. Στο σύστημα υπάρχουν οι βασικοί κατάλογοι που είναι υπεύθυνοι για την παροχή των βασικών λειτουργικών μονάδων της εφαρμογής, όπως:

- Controllers
- Models
- Repositories
- Services

αλλά και οι κατάλογοι που παρέχουν βοηθητικές λειτουργίες και επιλογές διαμόρφωσης του συστήματος, όπως:

- Configurations
- Enums
- Utils

Στους καταλόγους Controllers, Models, Repositories και Services υπάρχουν οι μονάδες του συστήματος για την υποστήριξη μιας παραλλαγής του μοντέλου ανάπτυξης MVCS (Model – View – Controller – Service), ενώ στον κατάλογο Configurations έχει οριστεί η μονάδα του συστήματος CorsConfiguration που παρέχει τις επιλογές διαμόρφωσης του συστήματος για την πολιτική CORS (Cross – Origin Resource Sharing) και στους καταλόγους Enums και Utils οι βοηθητικές μονάδες AnalysisMetrics και δομές δεδομένων, αντίστοιχα.

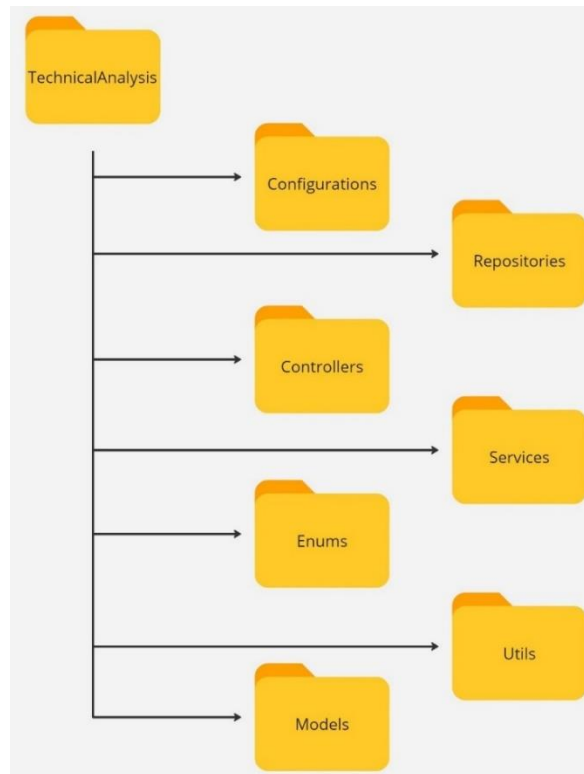


Figure 20: Spring Boot application's directory structure

Το σχεδιαστικό πρότυπο MVC (Model – View – Controller) είναι ένα από τα πιο διάσημα σχεδιαστικά πρότυπα στον τομέα της ανάπτυξης εφαρμογών. Το MVC περιγράφεται από τα τρία βασικά στοιχεία του, τα Models, τις Views και τους Controllers. Το πρότυπο, αυτό, έχει γίνει η βάση για αρκετές παραλλαγές, όπως το πρότυπο MVCS με την προσθήκη των Services. Το μοντέλο MVCS (Model – View – Controller – Service) είναι ακόμη ένα σχεδιαστικό μοντέλο βασισμένο στο MVC και το επεκτείνει με το στοιχείο των Services. Το στοιχείο των Services περιέχει τη λειτουργικότητα που αποτελεί τον πυρήνα της εφαρμογής. Η ανάπτυξη του εργαλείου έγινε με μια παραλλαγή αυτού του σχεδιαστικού προτύπου το στοιχείο Views, καθώς την προβολή των δεδομένων στον τελικό χρήστη αναλαμβάνει το Front End του εργαλείου.

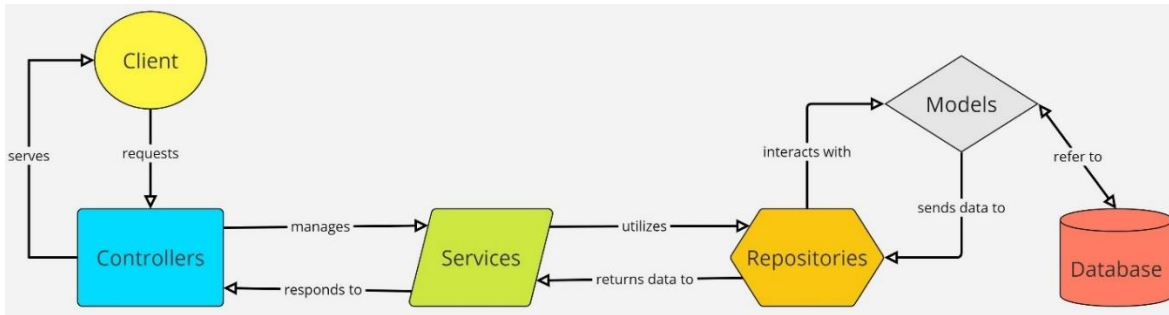


Figure 21: Spring Boot application's design pattern

Με τον όρο Model δηλώνουμε τις οντότητες που δημιουργεί, επεξεργάζεται και αποθηκεύει ένα σύστημα. Είναι το κύριο στοιχείο του μοντέλου, καθώς έχει την ευθύνη για τη διαχείριση των δεδομένων στο σύστημα αλλά και στην επικοινωνία του με τη βάση των δεδομένων. Με άλλα λόγια, τα Models είναι οι δυναμικές δομές δεδομένων της εφαρμογής και αλληλεπιδρούν τόσο με το ίδιο το σύστημα όσο και με το σύστημα της βάσης δεδομένων. Αντιπροσωπεύουν τις πληροφορίες που διαχειρίζεται η εφαρμογή. Στο εργαλείο, τα Models είναι οι οντότητες:

- Project
- Contributor
- Commit
- File

Οι οντότητες αυτές αντιπροσωπεύουν τις πληροφορίες που σχετίζονται με το έργο λογισμικού, τους κατασκευαστές τους λογισμικού, τις εκδόσεις και τις αλλαγές στο λογισμικό και τα αρχεία του λογισμικού, αντίστοιχα. Με τα δεδομένα αυτά το σύστημα του εξυπηρετητή μπορεί να αποθηκεύσει και να επεξεργαστεί αποτελεσματικά τα δεδομένα κάθε αποθετηρίου και να διαθέσει την πληροφορία αυτή μέσω των endpoints της. Για την αποθήκευση αυτών των πληροφοριών στη βάση δεδομένων μεσολαβεί το ενσωματωμένο σύστημα ORM της Spring Boot με την αξιοποίηση των Repositories. Κάθε Repository αντιστοιχίζεται σε μια από τις βασικές οντότητες και παρέχει ένα ολοκληρωμένο API τόσο για την αποθήκευση όσο και για την ανάκτηση των δεδομένων. Κάθε Service χρησιμοποιεί ένα ή περισσότερα από τα Repositories για την άμεση διάδρασή του με τη βάση και τις δομές δεδομένων.

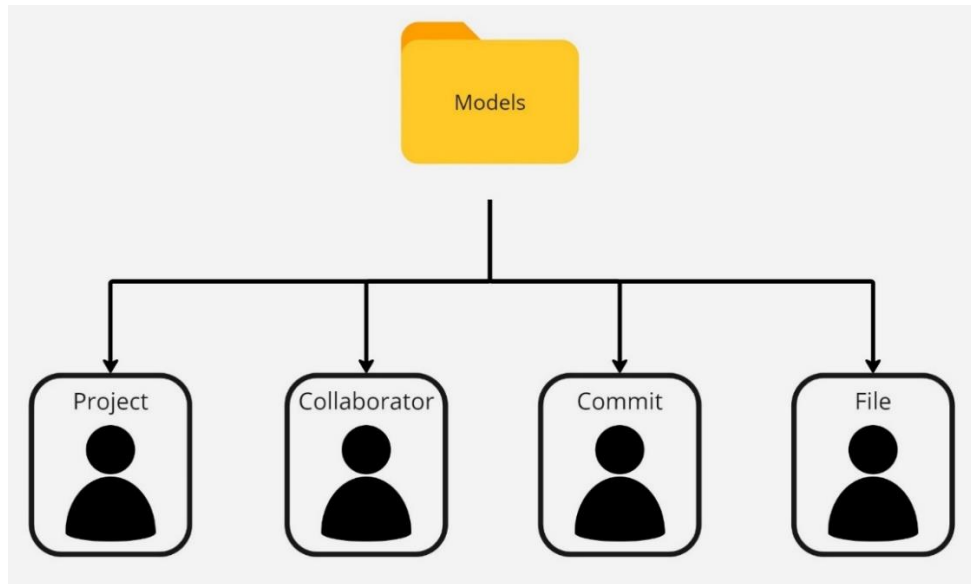


Figure 22: Spring Boot application's Models directory

Τα `Services` αποτελούν το `business logic` στον πυρήνα της εφαρμογής. Στα `Services` εκτελούνται οι λειτουργίες για την προσπέλαση, αποθήκευση και επεξεργασία των δεδομένων και πληροφοριών της εφαρμογής. Αναλυτικά, έχουν αναπτυχθεί τα κατάλληλα `Services` για την αρχικοποίηση ενός έργου λογισμικού, την εκτέλεση αιτημάτων σε RESTful εφαρμογές, την εκτέλεση εντολών CLI, την ανάλυση του λογισμικού και την εξαγωγή των αρχείων των αποτελεσμάτων.

Initialization Service

Το `Service` που είναι υπεύθυνο για την αρχικοποίηση του λογισμικού είναι ο κεντρικό «αγωγός» του συστήματος, καθώς αρχικοποιεί και χειρίζεται τα υπόλοιπα `Services`. Περιλαμβάνει πέντε μεθόδους εκ των οποίων η μια έχει δημόσια πρόσβαση ενώ οι υπόλοιπες τέσσερις παρέχουν εσωτερική λειτουργικότητα. Η δημόσια μέθοδος χρησιμοποιείται για την εκκίνηση της αρχικοποίησης ενός αποθετηρίου. Στην αρχή, πραγματοποιεί την αντιγραφή του αποθετηρίου τοπικά, με τη βοήθεια της υπηρεσίας `GitCLI`. Στη συνέχεια, βρίσκει τους `authors` και τα `commits` που έχουν γίνει συμπεριλαμβάνοντας και τα αρχεία που κάθε `commit`, μέσω του `GitCLI`, και αποθηκεύει τις πληροφορίες αυτές στα μοντέλα `GitHubCollaborator`, `GitHubCommit` και `GitHubFile`, αντίστοιχα, με τη χρήση των υπηρεσιών `GitLogReader` και `GitLogInterpreter`.

Εφόσον, πλέον, υπάρχουν όλες οι απαραίτητες δομές στο σύστημα, εκτελείται το φιλτράρισμα των commit σύμφωνα με την ημερομηνία δημιουργίας τους. Το φιλτράρισμα αποτελεί μέρος της βοηθητικής δομής GitHubCommitList και βρίσκει τα commit προς ανάλυση. Τα commit προς ανάλυση είναι εκείνα τα commit που δημιουργήθηκαν στο τέλος μιας εβδομάδας και αποτελούν την τελική μορφή του έργου για τη συγκεκριμένη εβδομάδα. Έπειτα, εκτελείται η ανάλυση στα commit τα οποία ικανοποιούν το φιλτράρισμα από την υπηρεσία SonarAnalysis. Τέλος, δημιουργείται το GitHubProject βάση του πιο πρόσφατου commit και αποθηκεύονται στην βάση οι οντότητες GitHubCollaborators, GitHubCommits, GitHubProject και GitHubFiles που έχουν δημιουργηθεί. Επίσης, στο σύστημα δημιουργείται και αποθηκεύεται το αρχείο .csv για το συγκεκριμένο αποθετήριο που περιέχει όλες τις πληροφορίες που για το έργο που ανακτήθηκαν κατά την αρχικοποίησή του.

Sonar Analysis Service

Η υπηρεσία SonarAnalysis αναλαμβάνει την πραγματοποίηση της ανάλυσης του έργου και την αποθήκευση των παραγόμενων αποτελεσμάτων στα μοντέλα του συστήματος. Η συγκεκριμένη υπηρεσία αποτελείται από εννέα μεθόδους εκ των οποίων οι δύο είναι δημόσια προσβάσιμες από το σύστημα, ενώ οι υπόλοιπες επτά αποτελούν εσωτερικές λειτουργίες της υπηρεσίας. Οι δύο δημόσιες μέθοδοι είναι υπεύθυνες για την αρχικοποίηση των παραμέτρων της ανάλυσης και την έναρξη της ανάλυσης. Η ανάλυση αρχικοποιείται με το όνομα του ιδιοκτήτη του έργου, το όνομα του αποθετηρίου, τον σύνδεσμο και τα διαπιστευτήρια του SonarQube. Η εκκίνηση της ανάλυσης αποτελείται από τέσσερα επιμέρους βήματα. Στο πρώτο βήμα, το σύστημα εκτελεί checkout στο commit το οποίο επιλέχθηκε για ανάλυση. Με αυτή τη διαδικασία το VCS εμφανίζει το σύστημα αρχείων, του έργου, όπως ήταν τη στιγμή που δημιουργήθηκε το commit. Στο δεύτερο βήμα, η υπηρεσία δημιουργεί το ειδικό αρχείο sonar-project.properties στην ρίζα του έργου, στο οποίο καθορίζονται οι επιλογές για την ανάλυση του συγκεκριμένου έργου. Στο τρίτο βήμα, το σύστημα εκτελεί το αρχείο sonar-scanner-*/bin/sonar-analysis.bat το οποίο ξεκινά την ανάλυση του καταλόγου του έργου στο SonarQube. Τέλος, στο τέταρτο βήμα, η υπηρεσία διενεργεί ανάκτηση των αποτελεσμάτων της ανάλυσης και αποθηκεύει τις πληροφορίες αυτές στα κατάλληλα μοντέλα.

Git CLI Service

Η υπηρεσία GitCLI παρέχει λειτουργικότητα για την εκτέλεση εντολών Git στη γραμμή εντολών του λειτουργικού συστήματος. Υλοποιεί τρεις μεθόδους οι οποίες είναι δημόσια προσβάσιμες από το σύστημα. Οι τρεις μέθοδοι είναι υπεύθυνες για την κλωνοποίηση ενός αποθετηρίου, την ανάκτηση του ιστορικού των commits σε κατάλληλα μορφοποιημένη έξοδο και την ανάκτηση των προγραμματιστών που συμμετείχαν στην ανάπτυξη του λογισμικού. Η εκτέλεση των εντολών γίνεται μέσω της ιεραρχίας στην οποία ανήκει, με στοιχείο – ρίζα το SimpleCLI. Το SimpleCLI είναι μια αφηρημένη κλάση η οποία διαθέτει μεθόδους και ιδιότητες για την επιτυχή εκτέλεση εντολών μέσω της γραμμή εντολών του λειτουργικού συστήματος.

Git Log Services

Για την ανάγνωση και διερμηνευση των αποτελεσμάτων των υπηρεσιών CLI χρησιμοποιούνται οι GitLogReader και GitLogInterpreter υπηρεσίες, αντίστοιχα. Οι υπηρεσίες αυτές εφοδιάζουν το σύστημα με μεθόδους κατάλληλες για την απορρόφηση των εξόδων των υπηρεσιών CLI και την ανάγνωση αυτών των ροών με την κατάλληλη διαδικασία και επεξεργασία, ώστε να δημιουργήσουν και να διαμορφώσουν τις δομές δεδομένων και τα μοντέλα πληροφοριών του συστήματος. Η υπηρεσία GitLogReader υλοποιεί δύο βασικές μεθόδους, δημόσια προσβάσιμες από το σύστημα. Οι μέθοδοι readCommits και readCollaborators δέχονται ένα αντικείμενο BufferedReader, που περιέχει το αποτέλεσμα μιας εντολής, και με συγκεκριμένη επεξεργασία του αντικειμένου επιστρέφουν εμφωλευμένες λίστες αλφαριθμητικών με τα δεδομένα των commits και των collaborators, αντίστοιχα. Στη συνέχεια, οι εμφωλευμένες δομές περνούν ως ορίσματα στις μεθόδους createCommitList και createCollaboratorList της υπηρεσίας GitLogInterpreter όπου και μετατρέπονται σε λειτουργικές μονάδες για το σύστημα με τη χρήση των βοηθητικών δομών δεδομένων GitHubCommitList και GitHubCollaboratorList.

Http Controller Service

Η υπηρεσία HttpControllerService χρησιμοποιείται για την ανάκτηση των αποτελεσμάτων από το SonarQube μέσω των διαθέσιμων web endpoints που παρέχει. Η υπηρεσία αποτελείται από μια εσωτερική κλάση, τη HttpRequest, και δύο μεθόδους τις getRequest και postRequest. Και οι δύο

μέθοδοι λαμβάνουν ως παράμετρο ένα αντικείμενο της εσωτερικής κλάσης `HttpRequest`, και με την αξιοποίηση της βιβλιοθήκης `Unirest` εκτελούν αιτήματα `GET` και `POST` επιστρέφοντας το `JSON` αντικείμενο της απάντησης του εξυπηρετητή.

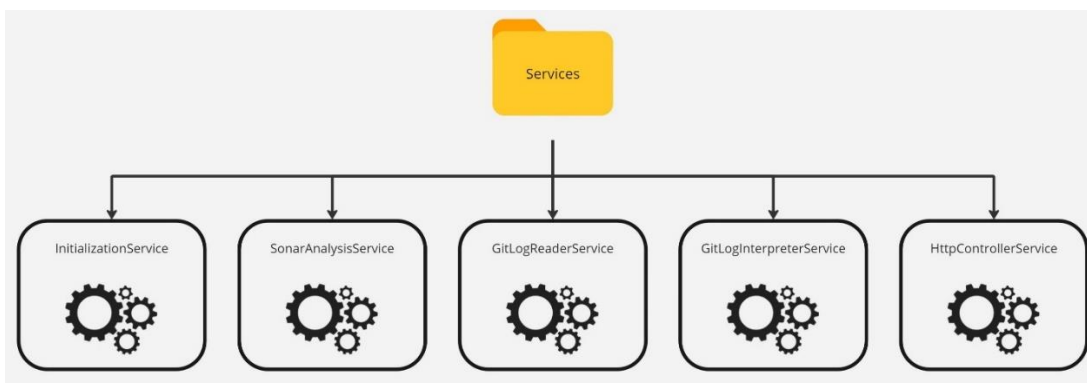


Figure 23: Spring Boot application's Services directory

Αυτές είναι οι βασικότερες υπηρεσίες, οι οποίες εκτελούν το μεγαλύτερο μέρος των λειτουργιών της εφαρμογής. Στο σύστημα έχουν οριστεί και άλλες υπηρεσίες οι οποίες ορίστηκαν με σκοπό να εξυπηρετούν τους διαθέσιμους `Controllers`. Οι υπηρεσίες `AnalysisFilesService`, `CollaboratorsService`, `CommitsService` και `FilesService` παρέχουν στους αντίστοιχους `Controllers` τη λειτουργικότητα που χρειάζονται για να εκτελέσουν τα αιτήματα που δέχονται. Αυτές οι υπηρεσίες έχουν ως κύριο σκοπό τη σύνδεση των `Controllers` με τη βάση δεδομένων, γι' αυτό ορίζονται με μεθόδους που εκτελούν κάποιο ερώτημα στην βάση των δεδομένων ώστε να επιστρέψουν το αποτέλεσμα στον `Controller`. Κάποιες μέθοδοι επιστρέφουν απλώς τα αποτελέσματα από τη βάση, ενώ κάποιες τα άλλες επεξεργάζονται για να δημιουργήσουν την απάντηση.

Παρ' ότι τα `Services` παρέχουν τις λειτουργίες στον πυρήνα του συστήματος, δεν έχουν άμεση επικοινωνία με τα αιτήματα του πελάτη. Τη σύνδεση αυτή κάνουν οι `Controllers`, οι οποίοι καθορίζουν ποιο ή ποια `Services` πρέπει να εκτελεστούν αφού λάβουν κάποιο εξωτερικό αίτημα. Οι `Controllers` είναι η διεπαφή του πελάτη με το σύστημα του εξυπηρετητή. Κάθε `Controller`

εξυπηρετεί μια διαδρομή URL. Η εφαρμογή αποτελείται από έξι Controllers υπεύθυνους για την υποστήριξη των REST των υπηρεσιών.

GET	/api/init	Initialize the application with a GitHub repository link	▼
GET	/api/export	Exporting project's results CSV file	▼
GET	/api/collaborators	Get all collaborators	▼
GET	/api/collaborators/{email}	Get collaborator by email	▼
GET	/api/projects	Get all projects	▼
GET	/api/projects/max-td	Get maxTd value of a project	▼
DELETE	/api/projects/delete	Delete a project	▼
GET	/api/commits	Get all commits	▼
GET	/api/commits/{sha}	Get commit by sha	▼
GET	/api/commits/grouped	Get commits grouped by date	▼
GET	/api/files	Get all files	▼
GET	/api/files/{name}	Get files by name	▼

Figure 24: Spring Boot application's web endpoints

Initialization Controller

Σε αυτόν τον Controller εξυπηρετούνται τα αιτήματα της διαδρομής `/api/init`. Παρέχει υλοποίηση μόνο για αιτήματα με τη μέθοδο GET και δέχεται ως μοναδική παράμετρο τον σύνδεσμο του αποθετηρίου ενός έργου. Το endpoint χρησιμοποιείται από το επίπεδο του πελάτη για να ξεκινήσει την αρχικοποίηση και ανάλυση ενός έργου λογισμικού.

Analysis Files Controller

Σε αυτόν τον Controller εξυπηρετούνται τα αιτήματα της διαδρομής */api/export*. Παρέχει υλοποίηση μόνο για αιτήματα με τη μέθοδο GET και δέχεται ως μοναδική παράμετρο το όνομα ενός έργου για το οποίο έχει γίνει η ανάλυση. Το endpoint χρησιμοποιείται από το επίπεδο του πελάτη για να επιστρέψει το αρχείο .csv της ανάλυσης ενός έργου.

Collaborators Controller

Σε αυτόν τον Controller εξυπηρετούνται τα αιτήματα της διαδρομής */api/collaborators*. Παρέχει υλοποίηση μόνο για αιτήματα με τη μέθοδο GET και μπορεί να δεχτεί μια παράμετρο μονοπατιού, το email κάποιου collaborator. Τα endpoints

- */api/collaborators*
- */api/collaborators/{email}*

χρησιμοποιούνται για την εύρεση

- όλων των collaborator
- ενός collaborator

, αντίστοιχα.

Projects Controller

Σε αυτόν τον Controller εξυπηρετούνται τα αιτήματα της διαδρομής */api/projects*. Παρέχει υλοποίηση για αιτήματα με τις μεθόδους GET και DELETE, και δέχεται το όνομα του έργου ως παράμετρο. Τα endpoints

- */api/projects*
- */api/projects/max – td*
- */api/projects/delete*

χρησιμοποιούνται για

- την εύρεση όλων των έργων
- την επιστροφή της μέγιστης τιμής TD σε ένα έργο
- τη διαγραφή ενός συγκεκριμένου έργου

, αντίστοιχα.

Commits Controller

Σε αυτόν τον Controller εξυπηρετούνται τα αιτήματα της διαδρομής */api/commits*. Παρέχει υλοποίηση μόνο για αιτήματα με τη μέθοδο GET, και μπορεί να δεχτεί το όνομα του έργου ή το αναγνωριστικό SHA του commit ως παράμετρο. Τα endpoints

- */api/commits*
- */api/commits/grouped*
- */api/commits/{sha}*

χρησιμοποιούνται για την εύρεση

- όλων των commits
- των commits ενός έργου με συγκεκριμένη ομαδοποίηση
- ενός συγκεκριμένου commit

, αντίστοιχα.

4.2.2. Data Layer

Το Data Layer αποτελείται από δύο docker container στους οποίους βρίσκονται οι βάσεις δεδομένων Neo4j και PostgreSQL. Η PostgreSQL χρησιμοποιείται από την υπηρεσία SonarQube, ενώ η Neo4j χρησιμοποιείται για την αποθήκευση των δεδομένων του συστήματος. Η Neo4j επιλέχθηκε καθώς είναι μια βάση δεδομένων που βασίζεται στους γράφους και έχει, μεταξύ άλλων, τη χαρακτηριστική δυνατότητα και την ευκολία της αναγνώρισης διάφορων μοτίβων που σχηματίζονται μεταξύ των συσχετίσεων των οντοτήτων της.

Οι οντότητες που αποθηκεύονται στη βάση δεδομένων Neo4j αντιστοιχούν στα μοντέλα της εφαρμογής Spring Boot, όπου:

- *GitHubProject* → *Project*
- *GitHubCollaborator* → *Collaborator*
- *GitHubCommit* → *Commit*
- *GitHubFile* → *File*

Επιπλέον, οι κόμβοι συσχετίζονται βάση των ιδιοτήτων που έχουν αποκτήσει στο αποθετήριο του έργου. Αναλυτικότερα, κάθε κόμβος Collaborator έχει μια συσχέτιση Contribute με κάποιον κόμβο Project, αφού ένας προγραμματιστής συμμετέχει στην ανάπτυξη ενός έργου λογισμικού, και μια συσχέτιση Committed_by με κάποιους κόμβους Commit, εφόσον δημοσίευσε τις αλλαγές του στο έργο. Τέλος, κάθε κόμβος Commit συσχετίζεται με κάποιους από τους κόμβους File, αφού οποιαδήποτε αλλαγή σε αρχείο περιέχεται σε ένα Commit, με τη σχέση Changed_in.

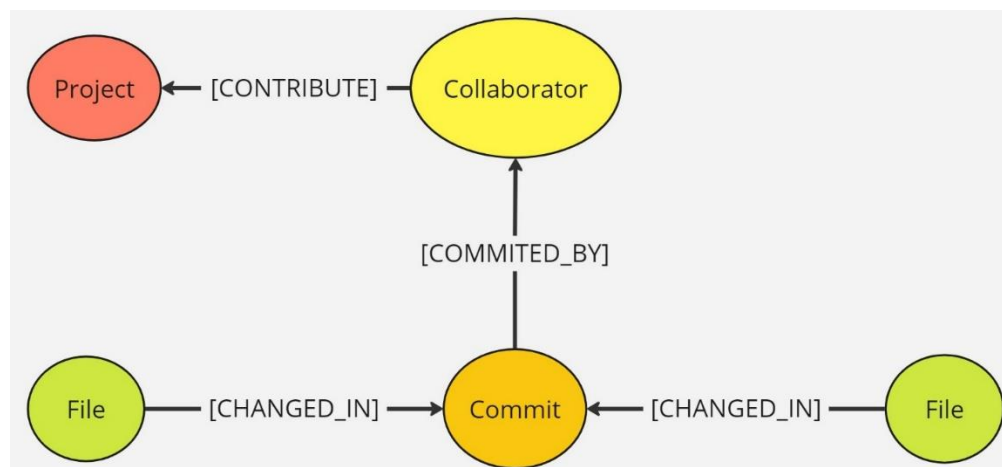


Figure 25: Neo4j database schema

Με αυτόν τον τρόπο, στη βάση δημιουργούνται γράφοι με διαδρομές από έναν κόμβο σε έναν άλλο, με συγκεκριμένες συσχετίσεις και ιδιότητες, αναπαριστώντας λεπτομερώς οποιοδήποτε αποθετήριο έργου λογισμικού.

4.3. Ανάπτυξη Frontend

Το Client Layer αναπτύχθηκε για την ευκολότερη διάδραση του χρήστη με το σύστημα. Είναι βασισμένο στην τεχνολογία React, χρησιμοποιώντας το περιβάλλον Node.js. Σε συνεργασία με τη βασική δομή της React αξιοποιήθηκαν οι βιβλιοθήκες Tailwind CSS και MaterialUI (ή MUI). Η βιβλιοθήκη Tailwind CSS προσαρμόζει μια ποικιλία προκαθορισμένων θεμάτων στα στοιχεία της εφαρμογής για την ευκολότερη τοποθέτηση και εμφάνιση τους στη σελίδα. Η βιβλιοθήκη MUI παρέχει στοιχεία ιστοσελίδων (π.χ. κουμπιά, πεδία, φόρμες κλπ.) τα οποία εμπεριέχονται σε ειδικά ορισμένες μονάδες συμβατές με εφαρμογές που χρησιμοποιούν React. Επιπλέον, η εφαρμογή χρησιμοποιεί, ως κύρια γλώσσα προγραμματισμού, τη TypeScript για την αποδοτικότερη λειτουργία της. Ο συνδυασμός αυτών των τεχνολογιών παράγει μια σύγχρονη και αξιόπιστη λύση για την ανάπτυξη διαδικτυακών εφαρμογών.

Η εφαρμογή στο επίπεδο του πελάτη σχεδιάστηκε ώστε να παρέχει στον χρήστη ένα φιλικό και εύκολο στην πλοήγηση περιβάλλον, με όλες τις απαραίτητες λειτουργίες. Αποτελείται από δύο βασικές σελίδες, τη σελίδα διαχείρισης (η οποία είναι και η κεντρική σελίδα) και τη σελίδα προβολής της ανάλυσης. Η πρώτη σελίδα περιέχει τον πίνακα διαχείρισης των έργων λογισμικού. Αποτελείται από δύο βασικά στοιχεία, το κουμπί προσθήκης ενός νέου έργου και τον πίνακα διαχείρισης των αναλυμένων έργων. Το κουμπί προσθήκης εμφανίζει στον χρήστη το αναδυόμενο μενού εισαγωγής όπου ο χρήστης εισάγει τον σύνδεσμο URL του αποθετηρίου και στη συνέχεια μπορεί να επιλέξει την έναρξη της ανάλυσής του. Στον πίνακα διαχείρισης, στο αριστερό μέρος, βρίσκονται τα ονόματα των έργων που έχουν αναλυθεί επιτυχώς από το σύστημα και στο δεξιό μέρος οι ενέργειες για την προβολή, εξαγωγή και διαγραφή ενός έργου. Οι ενέργειες λειτουργούν ως εξής:

- Η επιλογή της προβολής μεταφέρει τον χρήστη στη σελίδα ανάλυσης της εφαρμογής
- Η επιλογή της εξαγωγής ανακτά και κατεβάζει το αρχείο ανάλυσης .csv του έργου
- Η επιλογή της διαγραφής αφαιρεί την ανάλυση και τα σχετικά αρχεία του έργου από το σύστημα

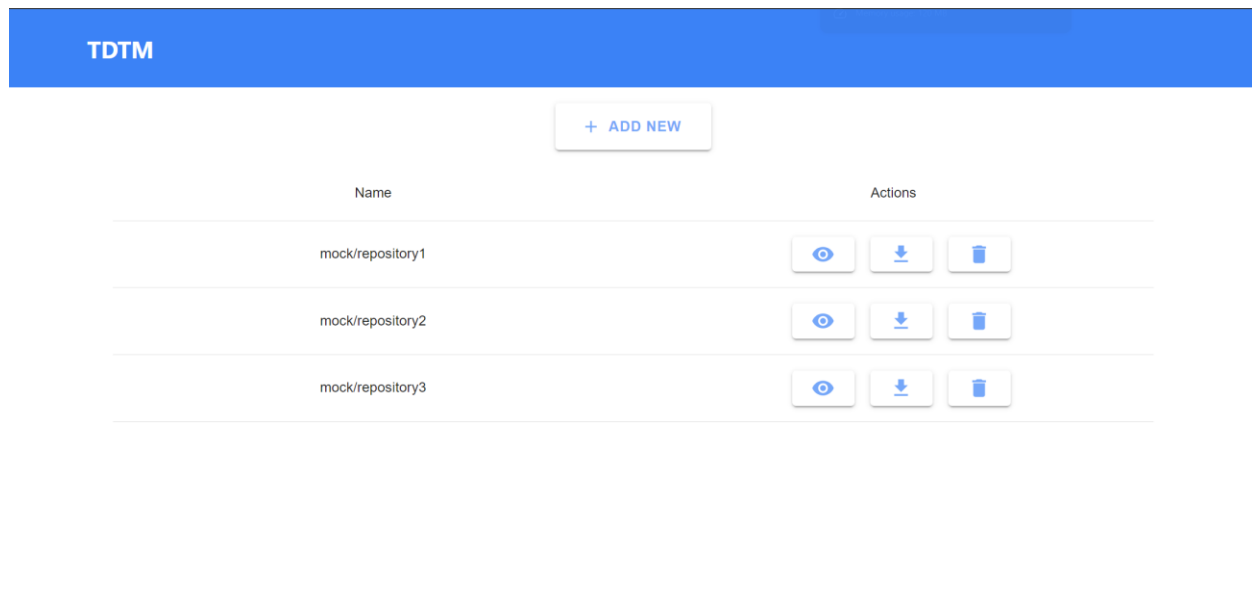


Figure 26: React JS application's Home page 1

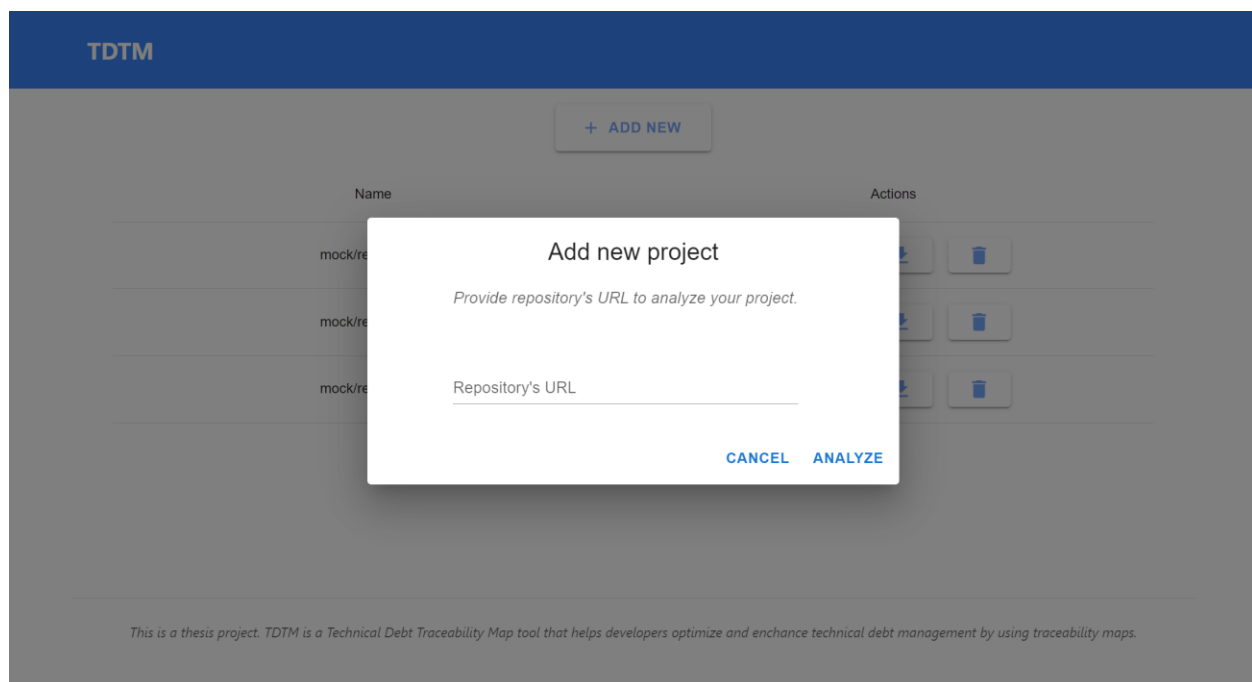


Figure 27: React JS application's Home page 2

Στη δεύτερη σελίδα της εφαρμογής, εμφανίζεται ο χάρτης ιχνηλάτισης του έργου λογισμικού. Αποτελείται από τον κεντρικό καμβά οπτικοποίησης, το panel των ιδιοτήτων, τον επιλογέα της μετρικής του τεχνικού χρέους και τον επιλογέα χρονικής περιόδου. Ο κεντρικός καμβάς αποτελεί μέρος της βιβλιοθήκης “vis-network” η οποία είναι μια επέκταση της βιβλιοθήκης vis.js για την υποστήριξη της React. Η βιβλιοθήκη αυτή παρέχει πλήρη υποστήριξη διαγραμμάτων οπτικοποίησης δεδομένων για διαδικτυακές εφαρμογές, έτσι χρησιμοποιήθηκαν τα διαγράμματα γράφων για τη δημιουργία των χαρτών ιχνηλασιμότητας. Στην τεκμηρίωση της βιβλιοθήκης υπάρχουν ποικίλες επιλογές και λειτουργίες για τη δημιουργία κόμβων και ακμών σε ένα δίκτυο. Οι κύριες επιλογές διαμόρφωσης για τη συγκεκριμένη εφαρμογή είναι:

- Layout
- Physics
- Color

Επιπλέον, ορίστηκαν μέθοδοι για την αλληλεπίδραση του χρήστη με τον χάρτη μέσα από τα συμβάντα που παρέχει το σύστημα της βιβλιοθήκης.

Ο γράφος αποτελείται από δενδρικές διατάξεις δύο επιπέδων, όπου στη ρίζα κάθε δένδρου εμφανίζονται οι προγραμματιστές και στα φύλλα τα αρχεία τα οποία αλλάχθηκαν την επιλεγμένη περίοδο από τον εκάστοτε προγραμματιστή. Τα φύλλα σε κάθε χρονική περίοδο χρωματίζονται με το εύρος RGB 0, 255, 255 – 255, 255, 255, ανάλογα με την τιμή της μετρικής του τεχνικού χρέους. Η τιμή του κόκκινου χρώματος διαμορφώνεται με τον εξής τύπο:

$$Color_R = \begin{cases} 0 & , TD_U = 0 \\ 255 - 255 * \frac{TD_F}{TD_U} & , TD_U \neq 0 \end{cases}$$

Όπου TD_U είναι η τιμή τεχνικού χρέους αναφοράς (η οποία εναλλάσσεται από τον χρήστη μέσω του επιλογέα τεχνικού χρέους) και TD_F είναι η τιμή τεχνικού χρέους του αρχείου για το οποίο υπολογίζεται η τιμή του κόκκινου χρώματος. Το panel των ιδιοτήτων εμφανίζεται στο άνω αριστερό μέρος του παραθύρου και αναλαμβάνει την προβολή των ιδιοτήτων του επιλεγμένου κόμβου. Τέλος, τα στοιχεία με τα οποία ο χρήστης μπορεί να αλληλεπιδράσει και να παραμετροποιήσει την οπτικοποίηση των δεδομένων είναι ο επιλογέας εναλλαγής του τεχνικού χρέους και ο επιλογέας της χρονικής περιόδου και βρίσκονται στο άνω δεξιό και στο κάτω μέρος

του παραθύρου, αντίστοιχα. Ο επιλογέας εναλλαγής του τεχνικού χρέους επιτρέπει στον χρήστη να εναλλάσσει την τιμή αναφοράς του τεχνικού χρέους, ώστε να χρωματίζει τα φύλλα – αρχεία κάθε ιεραρχικού δέντρου είτε σε σχέση με την περίοδο στην οποία ανήκουν είτε σε σχέση με όλη τη ζωή του λογισμικού. Δηλαδή, η τιμή της μετρικής του τεχνικού χρέους κάθε φύλλου – αρχείου ενός δέντρου συγκρίνεται είτε με υπόλοιπα φύλλα – αρχεία της περιόδου στην οποία ανήκει είτε με τα υπόλοιπα φύλλα – αρχεία όλων των περιόδων που παρέχονται στο σύστημα. Έτσι, ο χρήστης έχει τη δυνατότητα να οπτικοποιήσει τα δεδομένα του, ως προς τη μετρική του τεχνικού χρέους, επιτρέποντας του να λάβει μια αναλυτικότερη και πιο ακριβής εικόνα της εξέλιξης της μετρικής του τεχνικού χρέους στο σύστημα. Ο επιλογέας της χρονικής περιόδου επιτρέπει στον χρήστη να επιλέξει και να προβάλει την κατάσταση του λογισμικού σε μια συγκεκριμένη χρονική περίοδο, αυτή η επιλογή διευκολύνει την οπτικοποίηση καθώς περιορίζει τα διαθέσιμα δεδομένα σε συγκεκριμένες χρονικές περιόδους επιτρέποντας την καλύτερη κατανόηση τους από τον χρήστη.

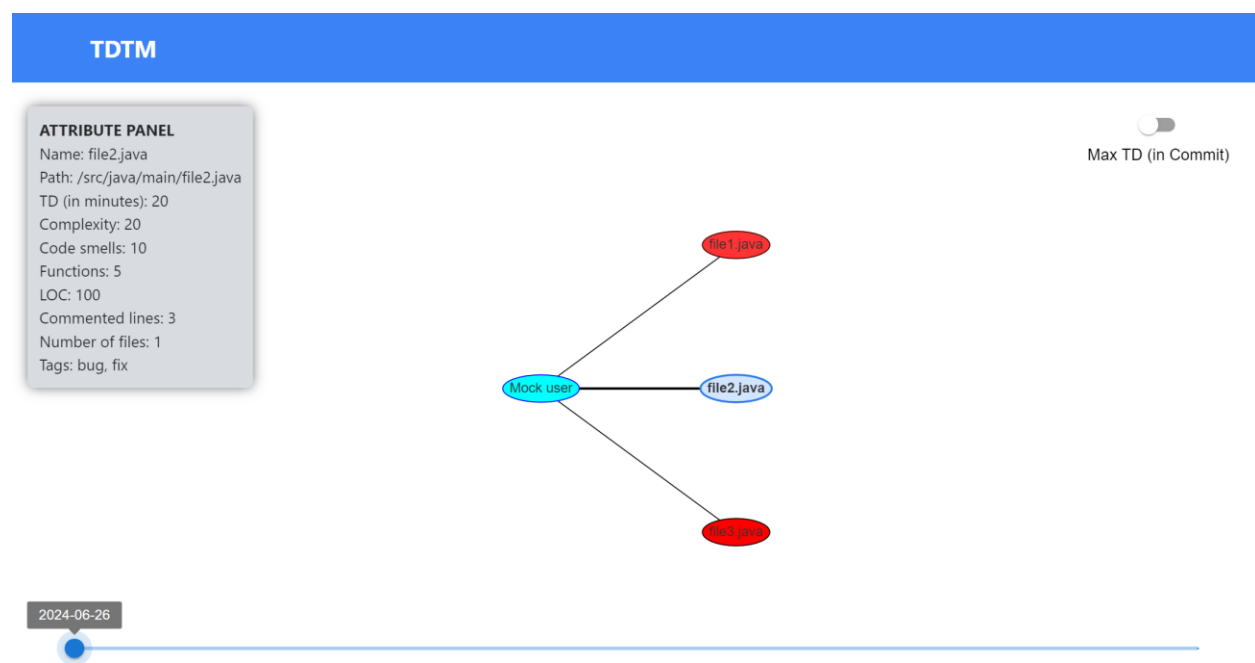


Figure 28: React JS application's Analysis page 1

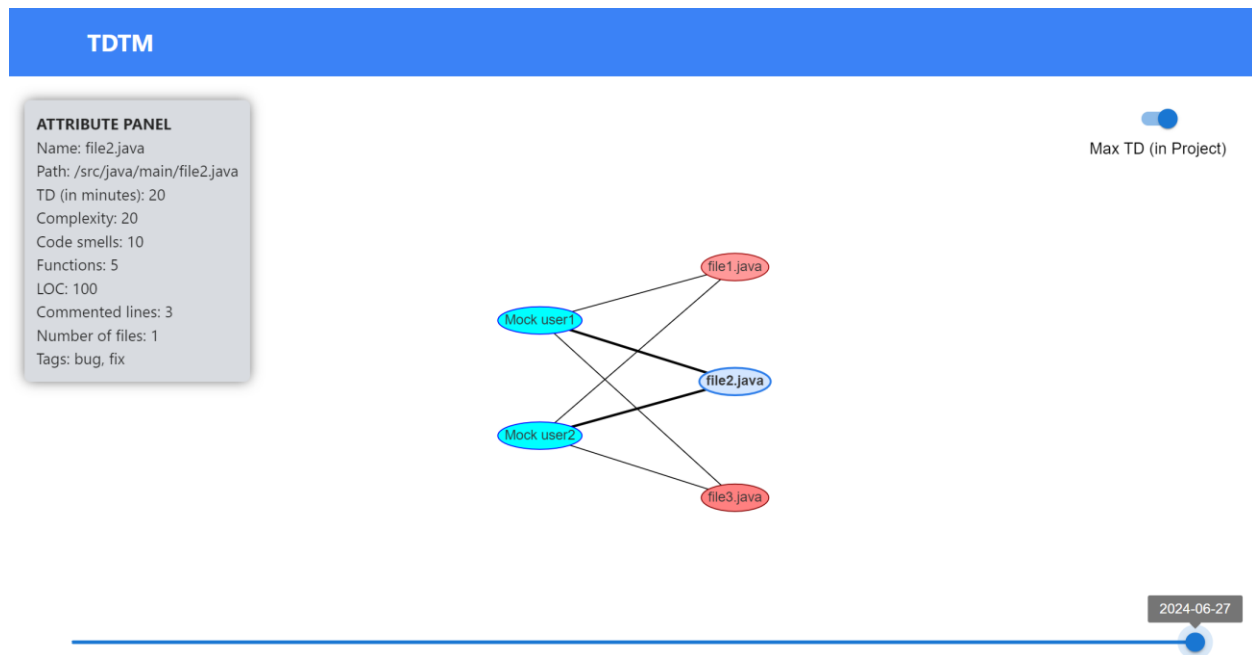


Figure 29: React JS application's Analysis page 2

Η εφαρμογή στο επίπεδο του πελάτη, είναι δομημένη σε καταλόγους με εννοιολογική βάση για κάθε επιμέρους λειτουργία του συστήματος. Στον κατάλογο στη ρίζα της εφαρμογής περιέχονται όλα τα απαραίτητα αρχεία και υποκατάλογοι όπως:

- build, κατάλογος που περιέχει όλα τα απαραίτητα στατικά αρχεία για τη διανομή της εφαρμογής
- node_modules, κατάλογος που περιέχει όλες τις εξωτερικές βιβλιοθήκες που χρειάζεται η εφαρμογή.
- public, κατάλογος που περιέχει τα στατικά αρχεία για τη λειτουργία της εφαρμογής.
- src, κατάλογος που περιέχει τα δυναμικά αρχεία για τη λειτουργία της εφαρμογής.
- .dockerfile & .dockerignore, αρχεία που χρησιμοποιούνται για τη δημιουργία του docker container της εφαρμογής.
- .gitignore, αρχείο που χρησιμοποιείται για το versioning της εφαρμογής
- package.json, ειδικό αρχείο που χρησιμοποιείται για καταγραφή metadata σχετικά με την εφαρμογή.

- `tailwind.config.js`, ειδικό αρχείο που χρησιμοποιείται για τη διαμόρφωση του πακέτου `css` της `tailwind`.
- `tsconfig.json`, ειδικό αρχείο που χρησιμοποιείται για την διαμόρφωση και τον ορισμό του υπερσυνόλου `TypeScript` στην εφαρμογή.

Στον κατάλογο `src`, περιέχονται όλα εκείνα τα αρχεία στα οποία στηρίζεται η λειτουργικότητα της εφαρμογής. Η δομή του συγκεκριμένου καταλόγου βασίζεται, επίσης, στην εννοιολογική συνοχή των στοιχείων τα οποία περιέχει. Περιέχονται, δηλαδή, τα εξής:

- `index.tsx` & `app.tsx`, αρχεία που σχετίζονται με την σύνδεση της εφαρμογής με το σύστημα `DOM`.
- `Components`, κατάλογος των επιμέρους ξεχωριστών στοιχείων που χρησιμοποιούνται από την εφαρμογή.
- `Pages`, κατάλογος των σελίδων πλοήγησης που υποστηρίζει η εφαρμογή.
- `Types`, κατάλογος των στατικών τύπων που χρησιμοποιεί η εφαρμογή.
- `Utils`, κατάλογος επιπρόσθετων αρχείων για τη διεκπεραίωση γενικευμένων λειτουργιών.

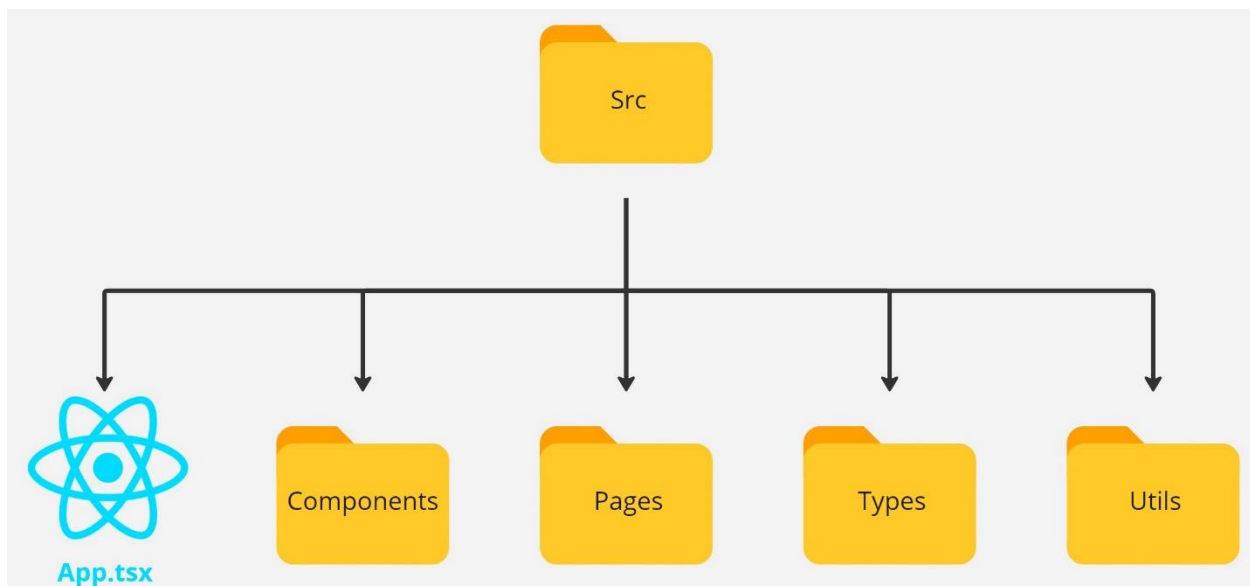
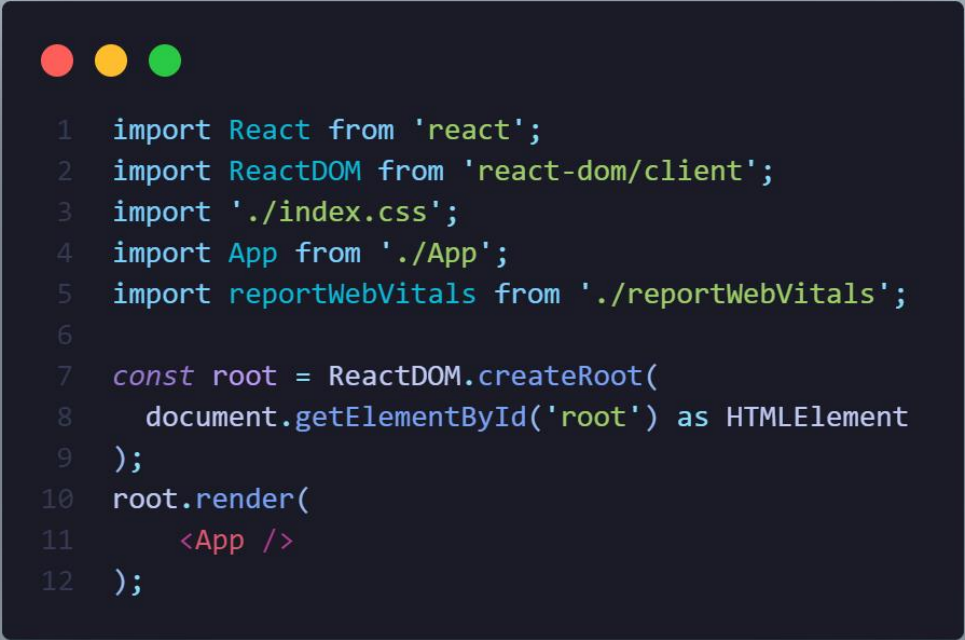


Figure 30: React JS application's source directory structure

Τα αρχεία `index.tsx` και `app.tsx` περιέχουν τον κορμό της διασύνδεσης του DOM με τα στοιχεία της React. Το βασικότερο από τα αρχεία αυτά είναι το `index.tsx` στο οποίο βρίσκεται το κεντρικό σημείο πρόσβασης των στοιχείων της React με τα στατικά αρχεία HTML. Σε αυτό, οι μηχανισμοί της React βρίσκουν και συνδέουν το HTML στοιχείο που φέρει την τιμή “root”, στην ιδιότητα `id`, με τον εικονικό DOM που διαχειρίζεται η βιβλιοθήκη, μέσω του στοιχείου – ρίζα `<App />`.

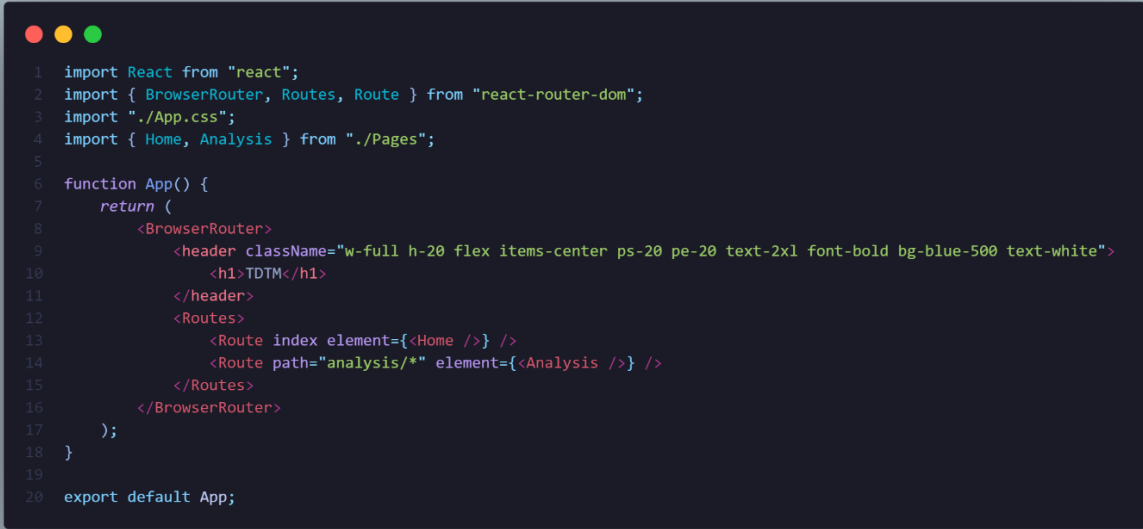


```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  const root = ReactDOM.createRoot(
8    document.getElementById('root') as HTMLElement
9  );
10 root.render(
11   <App />
12 );
```

Figure 31: React JS `index.tsx` file

Το στοιχείο `<App />` είναι ένα ειδικό συστατικό της React το οποίο τοποθετείται στη ρίζα του εικονικού DOM. Το στοιχείο αυτό ορίζεται στο αρχείο `App.tsx` και επιστρέφει ένα υποδέντρο το οποίο στο πρώτο επίπεδο έχει το στοιχείο `<BrowserRouter />`, στο δεύτερο επίπεδο τα στοιχεία `<header />` και `<Routes />` και στο τρίτο επίπεδο τα στοιχεία `<h1 />` και `<Route />`. Το στοιχείο `<header />` αντιπροσωπεύει την κεφαλίδα της εφαρμογής, η οποία εμφανίζεται σε όλες τις σελίδες της. Τα στοιχεία `<BrowserRouter />`, `<Routes />` και `<Route />` αποτελούν μέρος της βιβλιοθήκης “react-router-dom” και δίνουν στην εφαρμογή τη

δυνατότητα της περιήγησης, έτσι ορίζονται οι δύο διαδρομές στην εφαρμογή. Η πρώτη σελίδα βρίσκεται στη ρίζα της διαδρομής URL και αντιστοιχίζεται με το συστατικό `< Home />`, ενώ η δεύτερη σελίδα βρίσκεται στη διαδρομή `«/analysis/*»` (όπου ο χαρακτήρας `«*»` είναι μπαλαντέρ) και αντιστοιχίζεται με το συστατικό `< Analysis />`.



```
1 import React from "react";
2 import { BrowserRouter, Routes, Route } from "react-router-dom";
3 import "./App.css";
4 import { Home, Analysis } from "./Pages";
5
6 function App() {
7   return (
8     <BrowserRouter>
9       <header className="w-full h-20 flex items-center ps-20 pe-20 text-2xl font-bold bg-blue-500 text-white">
10        <h1>TDTM</h1>
11      </header>
12      <Routes>
13        <Route index element={<Home />} />
14        <Route path="analysis/*" element={<Analysis />} />
15      </Routes>
16    </BrowserRouter>
17  );
18 }
19
20 export default App;
```

Figure 32: React JS App.tsx file

Στον κατάλογο Components βρίσκονται όλα τα υπόλοιπα συστατικά στοιχεία που απαρτίζουν στην διαδικτυακή εφαρμογή. Υπάρχουν υποκατάλογοι για κάθε συστατικό στοιχείο, όπως:

- Dashboard
- Modal
- Network
- Panel

και περιέχουν, αντίστοιχα, τα αρχεία:

- Dashboard.tsx
- AddNewProjectModal.tsx
- ProjectNetwork.tsx
- AttributePanel.tsx

Στον κατάλογο Pages βρίσκονται τα συστατικά στοιχεία Home.tsx και Analysis.tsx τα οποία αποτελούν τις δύο διαφορετικές σελίδες της εφαρμογής. Τα στοιχεία αυτά βρίσκονται στο πρώτο επίπεδο του εικονικού DOM, αμέσως μετά το στοιχείο – ρίζα App.tsx, και χρησιμοποιούν τα συστατικά στοιχεία από τον κατάλογο Components.

Στον κατάλογο Types βρίσκονται όλοι οι τύποι που δημιουργήθηκαν για τις ανάγκες και τη λειτουργία της εφαρμογής. Πιο συγκεκριμένα, διατίθενται τα αρχεία Entities.ts, GraphTypes.ts, TableRecordType.ts και FunctionType.ts, όπου ορίστηκαν στατικοί τύποι για τις οντότητες του συστήματος, τις οντότητες του διαγράμματος γράφων, τις εγγραφές του πίνακα διαχείρισης έργων και των ορισμάτων των συναρτήσεων της εφαρμογής.

Τέλος, στον κατάλογο Utils βρίσκονται οι βοηθητικές λειτουργίες που χρησιμοποιούν τα συστατικά στοιχεία της React. Οι βοηθητικές λειτουργίες του συστήματος είναι:

- η επικοινωνία με τον εξυπηρετητή μέσω REST αιτημάτων
- ο ορισμός των επιλογών για τη διαμόρφωση του διαγράμματος γράφων
- ο ορισμός πρόσθετων λειτουργιών εμφάνισης μέσω προσαρμοσμένου θέματος
- ο ορισμός μεθόδων για τη διεκπεραίωση συγκεκριμένων λειτουργιών

και περιέχονται στα αρχεία Callouts.ts, Options.ts, Theme.ts και Utils.ts, αντίστοιχα.

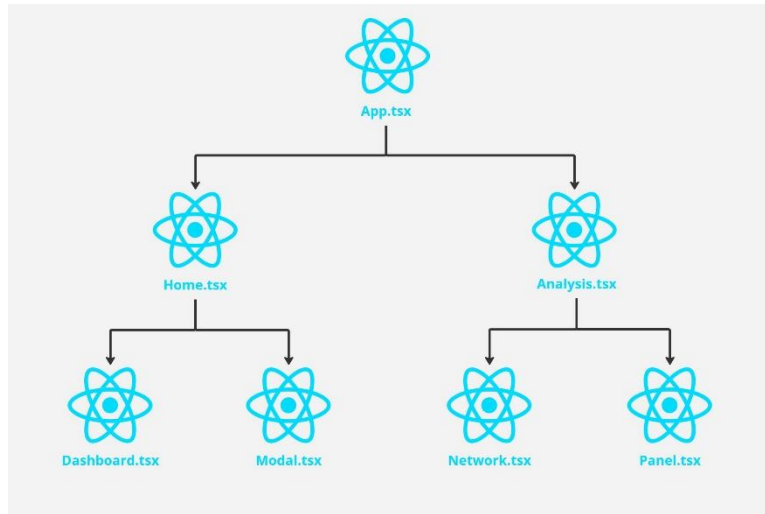


Figure 33: React JS virtual DOM

5. Επίλογος

Η παρούσα μελέτη επικεντρώθηκε στη δημιουργία και αξιολόγηση ενός εργαλείου Χάρτη Ιχνηλασιμότητας Τεχνικού Χρέους (Technical Debt Traceability Map Tool), με στόχο τη βελτίωση της διαχείρισης και οπτικοποίησης του τεχνικού χρέους σε έργα λογισμικού. Μέσα από τη διεξαγωγή μιας λεπτομερούς βιβλιογραφικής ανασκόπησης και την εφαρμογή σύγχρονων τεχνολογιών, επιτεύχθηκε η ανάδειξη της σημασίας και της αποτελεσματικότητας του εργαλείου.

Η έρευνα έδειξε ότι το εργαλείο βελτιώνει σημαντικά τη διαχείριση του τεχνικού χρέους, επιτρέποντας στους προγραμματιστές να έχουν μια σαφή εικόνα των αλληλεξαρτήσεων και των επιπτώσεων του τεχνικού χρέους στις διάφορες πτυχές του κώδικα. Τα αποτελέσματα των μελετών περίπτωσης καταδεικνύουν ότι η χρήση του εργαλείου όχι μόνο διευκολύνει την ανίχνευση και την παρακολούθηση του τεχνικού χρέους, αλλά και προάγει την καλύτερη κατανόηση και διαχείρισή του.

Παρόλο που η παρούσα μελέτη κατέγραψε σημαντικά ευρήματα, υπάρχουν περιορισμοί που θα μπορούσαν να εξεταστούν σε μελλοντική έρευνα. Ένας από τους βασικούς περιορισμούς είναι η ανάγκη για περαιτέρω εμπειρική αξιολόγηση του εργαλείου σε μεγαλύτερη κλίμακα και σε διάφορα περιβάλλοντα ανάπτυξης λογισμικού. Επιπλέον, η μελέτη επικεντρώθηκε κυρίως σε συγκεκριμένες τεχνολογίες (Java, Spring Boot, React, Neo4j), γεγονός που περιορίζει τη γενικότητα των αποτελεσμάτων.

Για το μέλλον, προτείνεται η επέκταση της μελέτης με την ενσωμάτωση και άλλων τεχνολογιών και πλαισίων (frameworks) ανάπτυξης λογισμικού, καθώς και η διεξαγωγή περαιτέρω εμπειρικών μελετών για την αξιολόγηση της αποτελεσματικότητας και της αποδοτικότητας του εργαλείου σε διαφορετικά περιβάλλοντα. Επιπλέον, η ανάπτυξη αυτόματων εργαλείων ανάλυσης και ειδοποίησης για τη διαχείριση του τεχνικού χρέους θα μπορούσε να συμβάλει στην περαιτέρω βελτίωση των διαδικασιών ανάπτυξης λογισμικού.

Συνοψίζοντας, η παρούσα μελέτη ανέδειξε τη σημαντικότητα της διαχείρισης του τεχνικού χρέους και την αναγκαιότητα για καινοτόμα εργαλεία που διευκολύνουν την παρακολούθηση και την αντιμετώπιση του. Το εργαλείο Χάρτη Ιχνηλασιμότητας Τεχνικού Χρέους αποτελεί ένα σημαντικό βήμα προς αυτή την κατεύθυνση, προσφέροντας πολύτιμες γνώσεις και πρακτικές λύσεις για τη βελτίωση της ποιότητας του κώδικα και τη διασφάλιση της βιωσιμότητας των έργων λογισμικού.

6. Αναφορές

- [1] Charalampidou, S., Ampatzoglou, A., Karountzos, E., & Avgeriou, P. (2020). Empirical studies on software traceability: A mapping study. *Journal of Software*, 33(2). <https://doi.org/10.1002/smr.2294>
- [2] Recovering traceability links between code and documentation. (n.d.). *IEEE Journals & Magazine | IEEE Xplore*. <https://ieeexplore.ieee.org/abstract/document/1041053>
- [3] Torkar, R., Gorschek, T., Feldt, R., Svahnberg, M., Raja, U. A., & Kamran, K. (2012). REQUIREMENTS TRACEABILITY: a SYSTEMATIC REVIEW AND INDUSTRY CASE STUDY. *International Journal of Software Engineering and Knowledge Engineering*, 22(03), 385–433. <https://doi.org/10.1142/s021819401250009x>
- [4] Achievements and challenges in State-of-the-Art Software Traceability between test and code artifacts. (n.d.). *IEEE Journals & Magazine | IEEE Xplore*. <https://ieeexplore.ieee.org/abstract/document/6862933>
- [5] BMC Software. (n.d.). Technical debt: The ultimate guide. *BMC Blogs*. Retrieved from <https://www.bmc.com/blogs/technical-debt/>
- [6] Cunningham, W. (1992). The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger*, 4(2), 29-30. Retrieved from https://en.wikipedia.org/wiki/Technical_debt
- [7] ProductPlan. (n.d.). Technical debt. Retrieved from <https://www.productplan.com/glossary/technical-debt/>
- [8] Asana. (n.d.). Technical debt. Retrieved from <https://asana.com/resources/technical-debt>
- [9] Tomar, R., & Kumar, S. (2024). Analyzing the concept of technical debt in the context of agile software development: A systematic literature review. *arXiv preprint arXiv:2401.14882*. Retrieved from <https://arxiv.org/pdf/2401.14882>
- [10] Woubshet Nema Behutiye a, Pilar Rodriguez a, Markku Oivo a, Ayşe Tosun b. (n.d.). Analyzing the concept of technical debt in the context of agile software development: A systematic literature review. *arxiv.org*. <https://arxiv.org/pdf/2401.14882>
- [11] A systematic review of requirement traceability techniques and tools. 2017 2nd International Conference on System Reliability and Safety (ICSRS) | 10.1109/ICSRS.2017.8272863. (n.d.). <https://scihub.se/https://ieeexplore.ieee.org/document/8272863>

- [12] Fangchao Tian, Tianlu Wang, Peng Liang, Chong Wang, Arif Ali Khan, Muhammad Ali Babar School of Computer Science, Wuhan University, Wuhan, China {tianfangchao, wangtianlu, liangp, cwang}@whu.edu.cn School of Computer Science, The University of Adelaide, Adelaide, Australia ali.babar@adelaide.edu.au Faculty of Information Technology, University of Jyväskylä, Finland arifaan_beghian@yahoo.com. (n.d.). The Impact of Traceability on Software Maintenance and Evolution: A Mapping Study <https://arxiv.org/>. <https://arxiv.org/pdf/2108.02133>
- [13] Achievements and Challenges in State-of-the-Art Software Traceability between Test and Code Artifacts. IEEE Transactions on Reliability, 63(4), 913–926 | 10.1109/TR.2014.2338254. (n.d.). <https://scihub.se/https://ieeexplore.ieee.org/abstract/document/6862933>
- [14] Nanette Brown, Yuanfang Cai, Yuepu Guo, Rick Kazman, Miryung Kim, Philippe Kruchten, Erin Lim, Alan MacCormack, Robert Nord, Ipek Ozkaya, Raghvinder Sangwan, Carolyn Seaman, Kevin Sullivan, Nico Zazworka. (n.d.). Managing Technical Debt in Software-Reliant Systems. insights.sei.cmu.edu. https://insights.sei.cmu.edu/documents/1434/2010_021_001_88118.pdf
- [15] Li, Z., Avgeriou, P., & Liang, P. (2015). A systematic mapping study on technical debt and its management. Journal of Systems and Software/the Journal of Systems and Software, 101, 193–220. <https://doi.org/10.1016/j.jss.2014.12.027>
- [16] Jane Cleland-Huang, Orlena C. Z. Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. 2014. Software traceability: trends and future directions. In Future of Software Engineering Proceedings (FOSE 2014). Association for Computing Machinery, New York, NY, USA, 55–69. <https://doi.org/10.1145/2593882.2593891>
- [17] Spanoudakis, G., & Zisman, A. (2005). SOFTWARE TRACEABILITY: a ROADMAP. In WORLD SCIENTIFIC eBooks (pp. 395–428). https://doi.org/10.1142/9789812775245_0014
- [18] Stack Overflow Developer Survey 2022. (n.d.). Stack Overflow. <https://survey.stackoverflow.co/2022/#section-version-control-version-control-systems>
- [19] About - Git. (n.d.). <https://git-scm.com/about>
- [20] Wikipedia contributors. (2024, June 7). Docker (software). Wikipedia. [https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))

- [21] Docker: Accelerated Container Application Development. (2024, May 20). Docker. <https://www.docker.com/>
- [22] Sonar. (n.d.). About - Sonar and SonarSource. <https://www.sonarsource.com/company/about/>
- [23] Wikipedia contributors. (2024, May 26). SonarQube. Wikipedia. <https://en.wikipedia.org/wiki/SonarQube>
- [24] Wikipedia contributors. (2024, June 14). Node.js. Wikipedia. <https://en.wikipedia.org/wiki/Node.js>
- [25] Node.js — about Node.js®. (n.d.). <https://nodejs.org/en/about>
- [26] Wikipedia contributors. (2024, June 17). React (JavaScript library). Wikipedia. [https://en.wikipedia.org/wiki/React_\(JavaScript_library\)](https://en.wikipedia.org/wiki/React_(JavaScript_library))
- [27] Wikipedia contributors. (2024, May 4). TypeScript. Wikipedia. <https://en.wikipedia.org/wiki/TypeScript>
- [28] ThemeSelection. (2023, September 26). JavaScript usage statistics that'll blow your mind...!! | By ThemeSelection | Medium | Quick Code. Medium. <https://medium.com/quick-code/javascript-usage-statistics-2023-thatll-blow-your-mind-2af8bce5a4f7>
- [29] Wikipedia contributors. (2024, May 19). JavaScript. Wikipedia. <https://en.wikipedia.org/wiki/JavaScript>
- [30] DB-Engines ranking. (n.d.). https://db-engines.com/en/ranking_trend/system/PostgreSQL
- [31] Wikipedia contributors. (2024, May 29). PostgreSQL. Wikipedia. <https://en.wikipedia.org/wiki/PostgreSQL>
- [32] PostgreSQL: about. (n.d.). The PostgreSQL Global Development Group. <https://www.postgresql.org/about/>
- [33] Postgres is the Undisputed Most Admired and Desired Database in Stack Overflow's 2023 Developer Survey. (n.d.). EDB. <https://www.enterprisedb.com/blog/postgres-most-admired-database-in-stack-overflow-2023>
- [34] Neo4j. (2020, March 30). Graph DBMS increased their popularity by 500% within the last 2 years - Graph Database & Analytics. Graph Database & Analytics. <https://neo4j.com/news/graph-dbms-increased-their-popularity-by-500percent/>
- [35] Neo4j. (2024, June 3). Native Graph Database | Neo4J Graph Database Platform. Graph Database & Analytics. <https://neo4j.com/product/neo4j-graph-database/>

- [36] Wikipedia contributors. (2024, May 29). Neo4J. Wikipedia. <https://en.wikipedia.org/wiki/Neo4j>
- [37] Wikipedia contributors. (2024, June 14). Spring boot. Wikipedia. https://en.wikipedia.org/wiki/Spring_Boot
- [38] Wikipedia contributors. (2024, June 13). Spring Framework. Wikipedia. https://en.wikipedia.org/wiki/Spring_Framework
- [39] Spring boot. (n.d.). Spring Boot. <https://spring.io/projects/spring-boot#overview>
- [40] TIOBE Index - TIOBE. (2022, June 3). TIOBE. <https://www.tiobe.com/tiobe-index/>
- [41] The top programming languages. (n.d.). The State of the Octoverse. <https://octoverse.github.com/2022/top-programming-languages>
- [42] Melanson, M. (2022, August 9). Don't call it a comeback: Why Java is still champ. GitHub. <https://github.com/readme/featured/java-programming-language>
- [43] Wikipedia contributors. (2024, June 11). Java virtual machine. Wikipedia. https://en.wikipedia.org/wiki/Java_virtual_machine
- [44] Wikipedia contributors. (2024, June 10). Object-oriented programming. Wikipedia. https://en.wikipedia.org/wiki/Object-oriented_programming
- [45] Wikipedia contributors. (2024, June 12). Java (programming language). Wikipedia. [https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language))
- [46] What is Java and why do I need it? (n.d.). https://www.java.com/en/download/help/whatis_java.html
- [47] Wikipedia contributors. (2024, April 25). Technical debt. Wikipedia. https://en.wikipedia.org/wiki/Technical_debt
- [48] Atlassian. (n.d.). Microservices vs. monolithic architecture | Atlassian. <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>
- [49] Wikipedia contributors. (2004, August 17). REST. Wikipedia. <https://en.wikipedia.org/wiki/REST>
- [50] Kumar, A. (2024, March 30). An Introduction to MVCS Architecture - Quantiphi. Quantiphi. <https://quantiphi.com/an-introduction-to-mvcs-architecture/>

- [51] Khalid, F. (2023, April 25). *How to conduct the design phase in software development*. DevTeam.Space. <https://www.devteam.space/blog/how-to-conduct-the-design-phase-in-software-development/>
- [52] Clark, H. (2024, June 3). *The Software Development Life Cycle (SDLC): 7 phases and 5 models*. The Product Manager. <https://theproductmanager.com/topics/software-development-life-cycle/>